

队伍编号	MC25005386
题号	B

基于 NSGA-II 的老城搬迁多目标优化决策研究

摘要

随着城市化进程的深入，城市更新成为提升城市内涵与利用存量空间的重要手段。老城区传统平房街区因产权分散和居民诉求多元，成为城市更新中的主要难题。传统搬迁模式难以兼顾居民安置与开发效益，亟需探索“平移置换”等兼顾双方利益的新路径。为解决以上问题，本文构建了多目标优化搬迁决策模型，采用**遗传算法（GA）**和**非支配排序遗传算法（NSGA-II）**对模型求解，并且提出了一种基于**自适应交叉变异概率函数**的改进遗传算法，能够有效提升算法效率，促进目标函数的适应度收敛。

针对问题一，构建了多因素加权的单目标搬迁方案决策模型，并采用改进的遗传算法进行求解。首先，构建了 3 个一级指标和 8 个二级指标，采用层次分析法确定各指标的权重，并对模型进行加权，以形成居民满意度函数。然后，以最大化居民满意度函数为目标，构建了混合整数优化模型，并以面积补偿、采光补偿和修缮补偿作为约束条件。接着，采用遗传算法求解该优化模型，为提高收敛速度并获得更优解，本文引入 sigmoid 函数改进交叉变异概率，显著提升了算法效率。

针对问题二，本文构建了一个多目标优化的老城区搬迁决策模型，并采用 **NSGA-II 算法**进行求解。该模型以最大化居民搬迁满意度、最小化搬迁住户数、最大化腾退整院面积为目标，生成了三维 Pareto 最优解集图，其中 49 个个体为前沿解。最终，计算了各方案的整院面积、收益与成本，结果显示，最大整院面积为 20002 平方米，最大盈利为 375214.3 万元，最高投入成本为 1794.4 万元，且解集内所有方案均可在预算内完成。

针对问题三，本文构建了性价比计算函数，以识别搬迁过程中的‘拐点’。首先，基于问题二得到的 Pareto 最优解集，计算了各解的性价比。然后，将性价比高于 20 的可行解作为初始种群，并追踪其目标函数值在每次变异后的变化。性价比拐点出现在个体 10 和个体 11 附近，拐点情况对应附件 10 中个体 ID 为 12 的搬迁方案，投入成本为

64.82 万元万元，整院面积为 20127 平方米，盈利为 373986 万元。

针对问题四，本文基于前文构建的模型和算法，设计了一套智能计算老城区平移置换决策的软件框架。该系统从住户、开发商和地块数据三方角度出发，设计了输入模块。软件架构包括数据预处理模块、满意度评价模块、优化求解模块等，能够实现地块和居民数据的自动读取。基于满意度评估和多目标优化逻辑，系统生成多种可选搬迁方案，并量化输出如整院腾退面积、搬迁户数、经济支出等核心指标。此外，系统还具备参数调节、方案对比和结果可视化功能，为管理部门在不同区域和政策背景下制定合理的搬迁方案提供支持。

关键词：多目标优化；改进 GA 算法；NSGA2 算法；帕累托解集

目 录

1.绪论	1
1.1 研究背景及意义	1
1.1.1 研究背景	1
1.1.2 研究意义	1
1.2 问题重述	2
1.3 技术路线图	4
2.模型假设与符号说明	5
2.1 模型假设	5
2.2 符号说明	5
3.问题一模型建立与求解	6
3.1 问题一分析	6
3.2 数据处理	6
3.3 模型建立	7
3.4 算法设计	9
3.4.1 遗传算法	9
3.4.2 改进遗传算法	11
3.5 模型求解	12
3.6 附加影响因素	14
3.6.1 层次分析法	14
3.6.2 模型建立	15
3.6.3 模型求解	17
4.问题二模型建立与求解	19
4.1 问题二分析	19
4.2 模型建立	19
4.2.1 基础概念定义	19
4.2.2 目标函数	20
4.3 NSGA-II 算法	20

4.4 搬迁方案结果计算	21
4.4.1 利益计算函数定义	21
4.4.2 帕累托解集	22
4.4.3 结果呈现	23
5.问题三模型建立与求解	24
5.1 问题三分析	24
5.2 性价比计算函数构建	24
5.3 结果分析	24
6.搬迁决策软件框架设计	26
6.1 问题四分析	26
6.2 软件设计与实现	26
6.2.1 系统功能架构	26
6.2.2 模块功能设计	27
7.模型评价与推广	29
7.1 模型评价	29
7.1.1 模型优点	29
7.1.2 模型不足	29
7.2 模型推广	29
参考文献	31
附录	32

1.绪论

1.1 研究背景及意义

1.1.1 研究背景

近年来，随着我国城市化进程进入深度发展阶段，城市建设已从“规模扩张”逐步迈向“内涵提升”的新阶段，针对于老旧小区的改造问题亟需解决^[1]。根据国家统计局数据，截至 2020 年，全国城镇化率已接近 64%，城市更新与存量空间的优化利用成为城镇发展转型的关键任务。特别是进入“十四五”以来，国家对老旧街区改造、历史风貌保护、居住品质提升等提出了更高要求，城市更新被赋予了更多社会与经济职能。

在这一背景下，老城区内的传统平房街区因其建筑形态独特、产权分散复杂、居民需求多元，成为城市更新实践中的重点关注对象。城市更新工程涵盖内容广泛，既包括城市生态环境的修复与功能体系的完善，也涉及历史文化遗产的保护与延续。同时，还包括基础设施的优化升级、老旧社区与城中村的系统改造等多个方面。这类更新项目投资空间巨大，正逐步成为推动城市高质量发展的新引擎和经济增长的重要动力^[2]。然而，传统“以整换整”或“一刀切式”搬迁方式，难以满足部分居民“留在原地”的生活诉求，导致出现“部分搬迁、部分留居”的共存局面。这类“混合使用”的空间形态不仅限制了整体院落的功能重构，也使得零散开发难以产生可观效益。因此，探索一种兼顾居民本地安置意愿与城市开发效益的“平移置换”模式，成为城市更新过程中的一项紧迫且现实的课题。从文献数量来看，国外在城市更新领域已有 3612 项研究成果，而国内的相关研究数量为 1878 篇，体现出全球在该领域的持续投入与探索^[3]。

1.1.2 研究意义

本研究围绕老旧街区内部居民“平移式搬迁”策略展开，结合实际居住情况与地块属性，通过构建科学合理的搬迁补偿机制与资源配置模型，旨在推动旧城片区的系统性优化和功能性重塑，具有以下几方面的研究价值和应用意义：

推动空间结构有序更新：通过在同一片区内部实现居民的重新布局与院落功能再分配，有助于腾出完整、连续的开发空间，提升片区整体可塑性与使用效率。实现多方利益平衡与协同优化：在保障居民居住舒适度和稳定性的基础上，最大化开发商的收益预期，探索开发商与居民共赢的城市更新模式。促进城市历史与人居环境的融合再造：相较于简单的“拆迁-置换”路径，“平移式”搬迁策略更加尊重原有的生活网络与文化。

综上所述，本文不仅为当前老城区更新过程中面临的“开发瓶颈”提供了一种可行的解决思路，也为未来全国范围内类似街区的搬迁实践提供了理论依据与技术支撑，具有显著的现实指导意义与长远的战略价值。

1.2 问题重述

在当前城市更新进程不断加快的背景下，如何科学、系统地推动老旧街区改造，已成为城市规划与管理实践中亟需破解的重要课题。尤其是在老城区中，传统平房杂院由于存在产权归属分散、历史遗留问题多、居民搬迁意愿差异显著等复杂因素，导致许多院落长期处于“部分搬迁、部分留居”的状态（如图 1-1 所示）。这种空间格局不仅破坏了街区整体的风貌连贯性与环境秩序，也在很大程度上削弱了土地的综合利用效率和后续开发潜力，形成“夹生街区”这一更新难点。

针对上述困境，规划实践中逐步提出并探索一种新型的搬迁策略，即“平移置换”模式。该模式在尊重居民原地居住意愿的基础上，充分挖掘街区内部的可用空间资源，通过将尚未搬迁的住户整体迁移至周边具备空置容量的地块，实现对现有分散杂乱空间的优化重组。

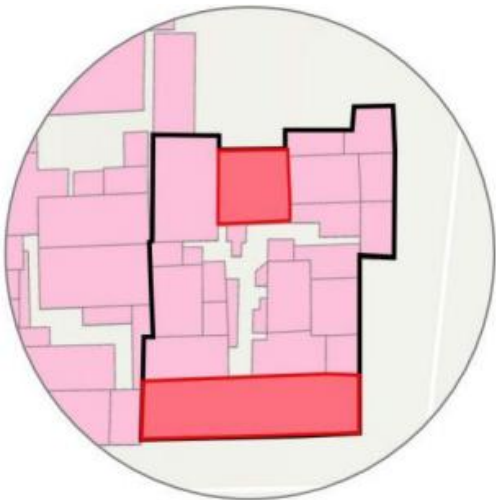


图 1-1 部分搬迁（粉色）和部分留居（红色）现象

为了提升居民参与搬迁的积极性，可采取三种补偿措施，具体见表 1。

表 1 补偿机制与标准说明表

补偿类型	补偿标准	说明
面积补偿	不低于原住房面积,最高为原面积的 130%	保证居住权益，受开发方预算限制
采光补偿	不劣于原居地块采光条件	朝向舒适度排序：南=北>东>西（见图 1-2）
修缮补偿	作为额外补偿手段实施修缮改造	面积与采光补偿无法满足时启用(上限为 20 万)

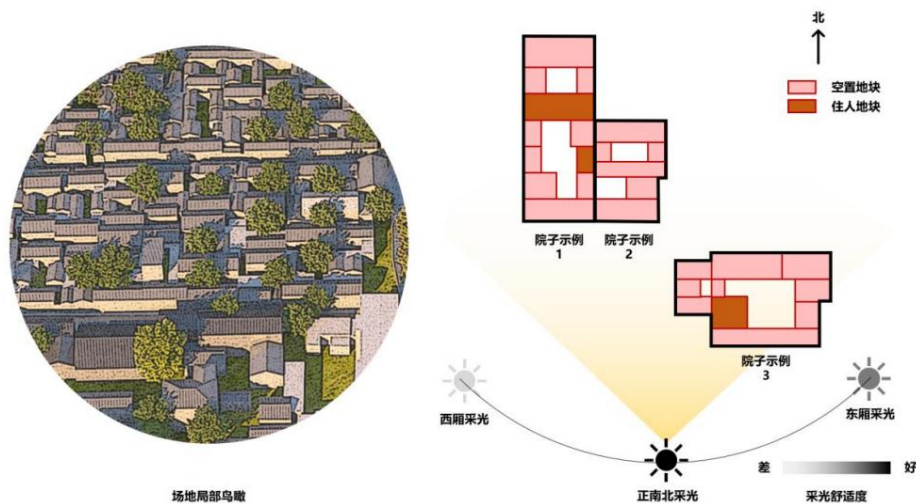


图 1-2 老城区场地局部鸟瞰及采光示意图

但该策略的实施并不简单，主要面临三个方面的难点：一是居民搬迁的接受度有限，尤其是当新地块在面积、朝向、居住舒适度等方面不具优势时；二是开发方搬迁投入存在预算上限，无法大规模提供额外补偿；三是街区内空置地块本身资源有限，难以兼顾所有住户的匹配需求。

围绕上述问题，根据题目提供的关于所研究的老街区的全部产权地块的基本情况主要包括地块 ID、地块所在的杂院 ID、地块的朝向、地块面积以及地块中是否有住户，结合相关的给定条件，本文将逐步开展以下几个方面的分析与建模：

问题 1：搬迁补偿机制与搬迁方案设计。建立一个合理的搬迁补偿规则体系，明确面积补偿和光照补偿作为硬性约束条件，并引入修缮投入作为弹性补偿手段，从而提升居民搬迁意愿，最终计算出从居民角度出发的最优搬迁方案。

问题 2：整院腾退方案优化。在尽量减少搬迁户数的前提下，优先腾退出最大面积、最大价值、且相互毗邻的完整院落，为开发商带来更高效益。该部分将围绕最大化居民满意度、最小化搬迁人数、最大化空置整院面积三个层面进行建模分析。

问题 3：搬迁投资回报评估。评估不同规模搬迁下的成本与收益变化趋势，判断是否存在性价比拐点——即进一步搬迁所带来的收益增长趋缓，甚至下降，从而为开发商确定最合适的搬迁方案。

问题 4：决策辅助系统构建。最后，尝试构建一套可推广的搬迁决策软件框架，明确关键参数、运算逻辑与输出指标，辅助相关部门在实际操作中实现快速规划与调整。

1.3 技术路线图

本文的技术路线具体见图 1-3：

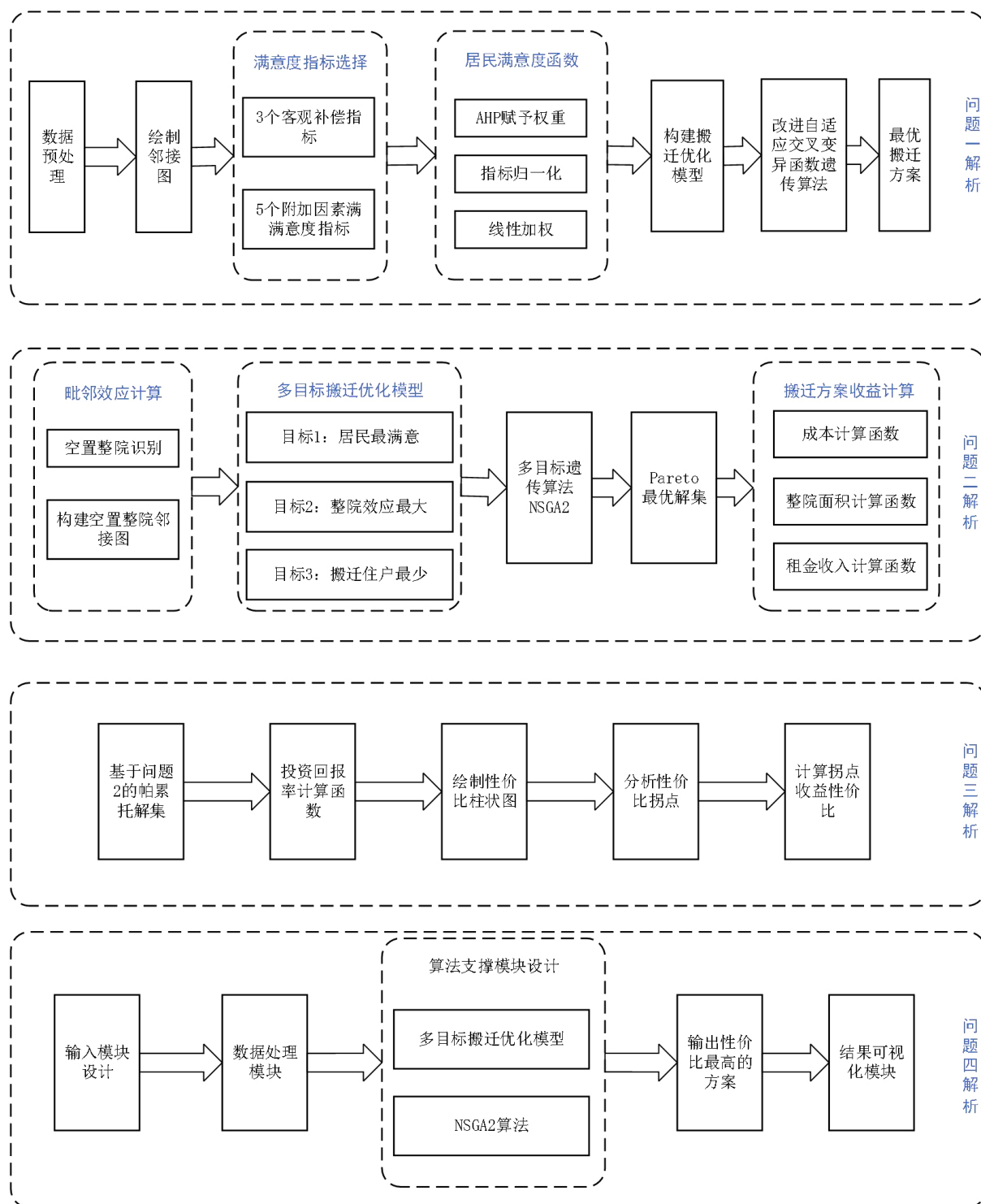


图 1-3 技术路线图

2.模型假设与符号说明

2.1 模型假设

假设 1：毗邻定义为两院落由边相接，对角相接的情况不视为毗邻；

假设 2：假设修缮单位面积成本按照当地的均价计算，本文取值为 2000 元/平方米；

假设 3：如果房屋进行了修缮翻新，居民就会同意搬迁；

假设 4：如果开发商不能为居民找到合适的搬迁地块，居民可以不用搬迁；

假设 5：不能进行二次搬迁，即居民只会搬向空房子，不存在原居民搬走而新居民搬入的情况，每人最多只能搬迁一次，地块最多只接受一次。

2.2 符号说明

本文中所用到的符号具体见表 2：

表 2 主要符号汇总表

符号	定义
状态变量	
i	居民原始居住地块编号， $i = 1, 2, \dots, 113$
j	可迁入地块编号， $j = 1, 2, \dots, 371$
k	院落 ID， $k = 1, 2, \dots, 107$
t_i	地块 i 是否有住户，0-1 变量
s_i	地块 i 面积
l_i	地块 i 的采光度， $l_i = 1, 2, 3$ ，分别代表西、东、南北三类采光
o_i	地块 i 的朝向，0-1 变量，0 表示东西朝向，1 表示南北朝向
V_k	院落 k 中原有居民的地块集合
$rcost$	单位翻新成本（单位：万元/平方米）
T	搬迁时间，取值为 $T = 120$ ，单位：天
x_{ij}	表示居民 i 是否搬到地块 j
r_j	是否修缮地块 j ，0-1 变量
z_k	院落 k 是否被完整腾空，0-1 变量

3.问题一模型建立与求解

3.1 问题一分析

首先，为制定科学有效的搬迁补偿方案，我们构建搬迁决策优化模型，根据居民当前居住状况，优化其搬迁后的接受意愿。该模型考虑居民重点关注的三个指标：面积补偿、采光补偿、修缮补偿，通过数据标准化消除量纲，并进行加权作为居民满意度函数。构建以居民满意度为目标函数的单目标混合整数优化模型，以面积补偿、采光补偿以及修缮补偿范围作为约束条件。

其次，我们采用遗传算法计算得到该优化模型的搬迁方案。为了得出更优结果并加快算法的收敛速度，我们提出一种改进的自适应交叉变异概率函数遗传算法，选用sigmoid 函数作为交叉变异概率函数，使得算法效率有效提升。

最后，为增强模型的适用性与解释力，我们还增加了 5 个影响搬迁意愿的隐性因素，包括新地块是否临近主街道、住户的密集程度、是否保持原有邻里关系等。同时考虑基础三个指标，采用层次分析法赋予 8 个指标权重，对数据进行标准化处理以消除量纲，然后进行线性加权作为目标函数，通过改进遗传算法得出居民视角的最优搬迁方案。

3.2 数据处理

1.数据预处理

根据题目附件一中给定的信息，我们对数据进行预处理，统计缺失或错误数据。然后对数据进行可视化，以便于初步分析问题。我们统计出各个院落的毗邻情况以及院落中居民居住情况的具体信息，红色圈代表该院落中所有地块都有居民居住，蓝色圈代表该院落中部分地块有居民居住，白色圈代表该院落中没有居民居住，具体见图 3-1。

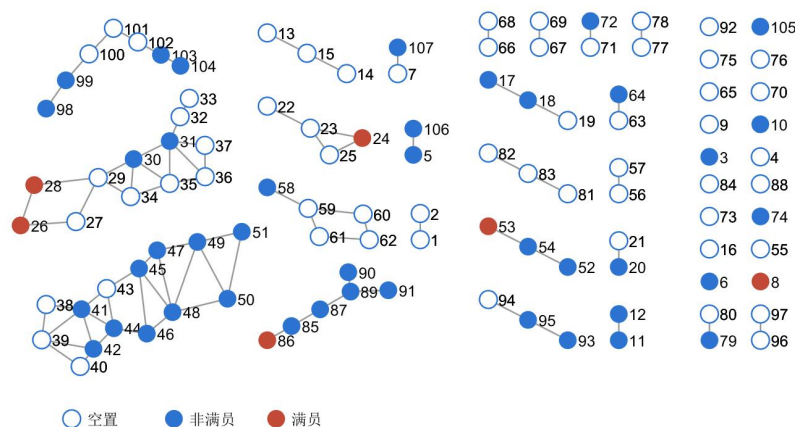


图 3-1 院落毗邻及居民居住情况

2.数据归一化处理

由于后文面积补偿、采光补偿以及修缮补偿的量纲不同，需要对数据进行归一化处理，本文采用线性归一化消除量纲影响，按照下式将数据缩放到区间[0,1]之间，便于后续权重的计算，提高计算结果的合理性。

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (3-1)$$

式中：

X ：原始数据

$X_{\min}$ ：原始数据中的最小值

$X_{\max}$ ：原始数据中的最大值

对于 IS_{ij} ，面积补偿满意度指标，其值等于搬入地块的面积减去搬出地块的面积，即 $IS_{ij} = s_j - s_i$ ，根据题目所给附件一，在满足前文提到的面积约束以及朝向约束的可行解中得到居民从地块 ID393 搬入地块 ID399 时能得出面积补偿的最大值为 $231-178=53 m^2$ ，居民不搬时得出面积补偿的最小值为 $0 m^2$ 。

对于 IL_{ij} ，采光补偿满意度指标，其值等于搬入地块的采光减去搬出地块的采光，即 $IL_{ij} = l_j - l_i$ ，根据前文可知，当从西向搬至南北向时，采光补偿最大值为 2，当居民不搬迁时得出采光补偿最小值为 0。

对于 RB_{ij} ，修缮补偿满意度指标，代表居民从地块 i 搬入地块 j 的修缮补偿金额，即 $RB_{ij} = r_j \cdot s_j \cdot rcost$ ，根据前文可知，每户居民搬迁时，对于迁入地块的翻新金额上限为 20 万元，即修缮补偿最大值为 20 万元，修缮补偿最小值为 0。

3.3 模型建立

根据前文对问题一的分析，本章主要站在居民的角度上进行分析，仅考虑居民搬迁的满意度，暂未考虑开发商的收益和成本，以居民的满意度为目标建立目标函数，建立混合整数优化模型。

1.目标函数

$$\max z = w_1 \sum_{i,j} x_{ij} \cdot IS'_{ij} + w_2 \sum_{i,j} x_{ij} \cdot IL'_{ij} + w_3 \sum_{i,j} x_{ij} \cdot RB'_{ij} \quad (3-2)$$

式中：

w_1 ：面积补偿所占的权重，取 0.6

w_2 : 采光补偿所占的权重, 取 0.3

w_3 : 修缮补偿所占的权重, 取 0.1

IS'_{ij} : 归一化后的居民从地块 i 搬去地块 j 的面积补偿

IL'_{ij} : 归一化后的居民从地块 i 搬去地块 j 的采光补偿

RB'_{ij} : 归一化后的居民从地块 i 搬去地块 j 的修缮补偿

3. 约束条件

如前文所述, 为促动居民搬迁, 开发商需要制定合理的搬迁激励机制, 所以在模型求解的过程中需考虑一定的约束使得问题更加符合常理, 针对问题一的约束具体如下。

(1) 面积补偿约束:

$$s_j \geq s_i \text{ and } 1.3 \cdot s_i \geq s_j \quad (3-3)$$

上式表明, 如果居民会从地块 i 搬至地块 j , 则必须满足迁入地块的面积不小于迁出地块的面积, 并且不大于迁出地块面积的 1.3 倍。

(2) 采光补偿约束:

$$l_j \geq l_i \quad (3-4)$$

上式表明, 如果居民会从地块 i 搬至地块 j , 则必须满足迁入地块的采光条件不低于迁出地块的采光条件, 具体以地块的朝向为判断标准。

(3) 修缮补偿约束:

$$r_j = \begin{cases} 1, & \text{if } s_i = s_j \text{ or } l_i = l_j \\ 0, & \text{else} \end{cases} \quad (3-5)$$

$$r_j \cdot s_j \cdot rcost \leq 20 \quad (3-6)$$

上式表明, 如果面积和采光补偿难以打动居民, 开发商可以通过修缮翻新迁入地块, 提高居住舒适度, 从而增强搬迁的吸引力。若开发商选择对迁入地块进行修缮翻新来推动居民搬迁, 则每迁入一户居民, 开发商在该地块修缮翻新方面的投入上限为 20 万元。

(4) 决策变量约束:

$$\sum_i x_{ij} \leq 1 \quad \text{and} \quad \sum_j x_{ij} \leq 1 \quad \forall i, j \quad (3-7)$$

上式表明, 每个地块 j 最多只能被一个居民 i 分配到, 每位居民只能搬到一个地块。

3.4 算法设计

3.4.1 遗传算法

本文构建的混合整数优化模型，为确定 113 位居民是否搬出，且搬入到哪个地块，需要包含 113×371 个决策变量，因此本文研究的搬迁优化问题为 NP-hard 问题。遗传算法（GA）是通过模仿生物界中染色体基因进化过程实现的自启发式算法，常用于求解车间调度、车辆路径优化、设施布局与分配、交通问题等 NP-hard 问题。其突出的全局搜索能力和良好的扩展性使其成为解决大规模问题的理想选择，因此本文采用遗传算法进行问题求解。传统遗传算法流程如图 3-2 所示：

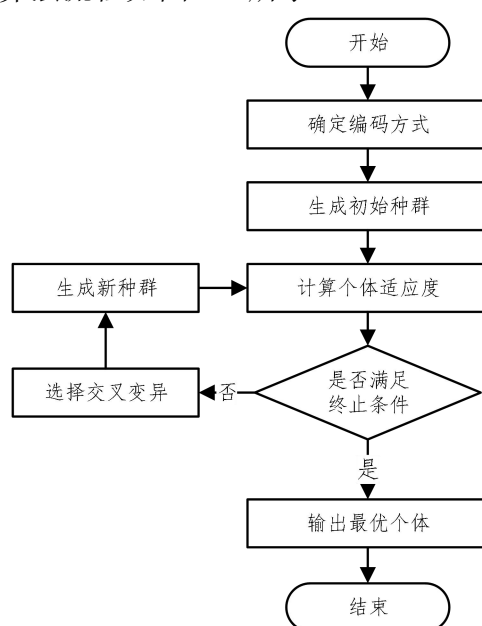


图 3-2 传统遗传算法流程图

结合选题的特点，以及遗传算法相对一些常规的优化算法，能够较快获得较好的优化结果的优势，主要采用遗传算法求解问题一，具体步骤如下：

1. 编码方式

本文共有 113×371 个决策变量，其中 113 代表有 113 户居民等待下一步的搬迁活动，371 代表居民有 371 个可能的地块选择。采用 0-1 矩阵编码需要创建 113×371 的矩阵，其中会有大量无用数据，造成储存冗余，我们采用实数编码方式，只需要构建 1×113 的行向量，其中第 i 个元素代表第 i 户原居民的搬迁意愿，0 表示不搬迁，1~371 表示搬迁至对应 ID 的地块。通过解码，我们可以得到原住民是否搬迁，以及搬入地块的 ID 的决策。因此，本文主要采用实数编码，不仅能提高算法的效率，便于大空间搜索，还便于处理复杂决策变量约束条件，适合组合优化问题^[4]。

2.初始种群

当初始种群中个体都是 0 时，即所有居民都不搬迁，这是一个可行解，因此，为了加快初始种群的生成效率，在满足约束条件下，我们采用稀疏编码的方式进行编码，使用少量非零元素来表示决策变量。为了加快初始可行解生成，我们选择个体中随机生成 100-110 个 0 元素，剩余元素为随机生成 1-371 中的随机整数，并经过约束条件检验，生成包含 50 个个体的初始种群。

3.计算适应度函数

通过计算适应度函数来评估每一个个体的优劣性，本章节所构建模型为最大化目标函数，因此直接采用目标函数作为适应度函数，适应度函数表示居民的满意度。

4.选择操作

本文采用一种非传统策略，根据个体的适应度从种群中随机选择 4 个个体，从这 4 个个体中选择最差的个体作为“父代”参与后续的交叉变异操作。采用这种方法主要是由于问题特性，对最优个体中 DNA 片段进行小幅修改也可能会造成新的个体违反约束条件，从而错失更优的解。选择最坏个体进行交叉变异，有助于跳出局部最优，引入新的基因组合，能使解集整体更优。

5.交叉操作

通过交换一对染色体的基因来产生后代，根据上述操作中选择的“父代”，然后从剩余的个体中随机选择一个作为“母代”，采取两点交叉的方法，通过随机确定两点的位置使“父代”和“母代”进行交叉操作，具体方法见下图。我们会对交叉操作产生的个体进行约束条件检验，若不能通过约束条件检验，则重新交叉循环直到通过约束。

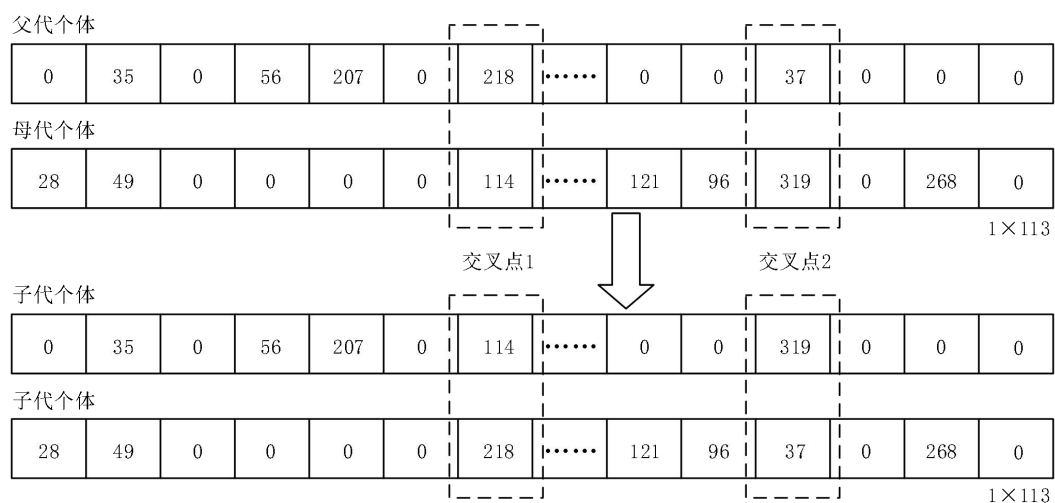


图 3-3 两点交叉示意图

6.变异操作

采用单点变异的方法，随机选择某个变异点并令其生成 0-371 之间的随机整数。某一个体经过变异操作之后，我们会判断其可行性，如果可行，则保留使用。

7.迭代与终止条件

本文将迭代上限设置为 40000 次，当迭代次数达到上限后，算法终止并输出最优解。

3.4.2 改进遗传算法

本文针对传统固定交叉概率的不足，参考湛文静、杨世忠、陈金广等人提出的自适应交叉变异概率^[5,6]，并根据本文模型中的最小化适应函数以及问题特性，提出一种改进的交叉变异概率函数。自适应变异交叉概率的方法可以根据遗传算法的表现来调整交叉和变异概率。当算法的表现较差时，可以增加变异概率，降低交叉概率；当表现较好时，可以增加交叉概率，降低变异概率^[7]。

1.改进交叉变异概率函数

本文选用 sigmoid 函数作为交叉变异概率函数，该函数在线性和非线性行为之间显现出较好的平衡，适合本文所建立的模型，sigmoid 函数简化方程如下式所示。当 $x \geq 9.903438$ 时， $\text{sig}(x)$ 的值接近于 1，当 $x < 9.903438$ 时， $\text{sig}(x)$ 的值接近于 0。

$$\text{sig}(x) = \frac{1}{1 + e^{-x}} \quad (3-9)$$

交叉概率变化函数：

$$P_c = \begin{cases} \frac{P_c^{\max} - P_c^{\min}}{1 + \exp\left(-c_0 + \frac{2c_0 f_{\text{avg}} - f_c^{\min}}{f_{\text{avg}} - f_c^{\min}}\right)} + P_c^{\min}, & f_c^{\min} < f_{\text{avg}} \\ P_c^{\max}, & f_c^{\min} \geq f_{\text{avg}} \end{cases} \quad (3-10)$$

式中 $P_c^{\max}$ 和 $P_c^{\min}$ 分别表示交叉概率的上界、下界， f_{avg} 表示种群适应值的平均值， $f_c^{\min}$ 表示参与交叉父代中适应值交友一方的适应值，当 $P_c^{\min}$ 取值为 0.5， $P_c^{\max}$ 取值为 0.8，交叉概率函数图像如图 3-4 所示，变异概率函数同理。

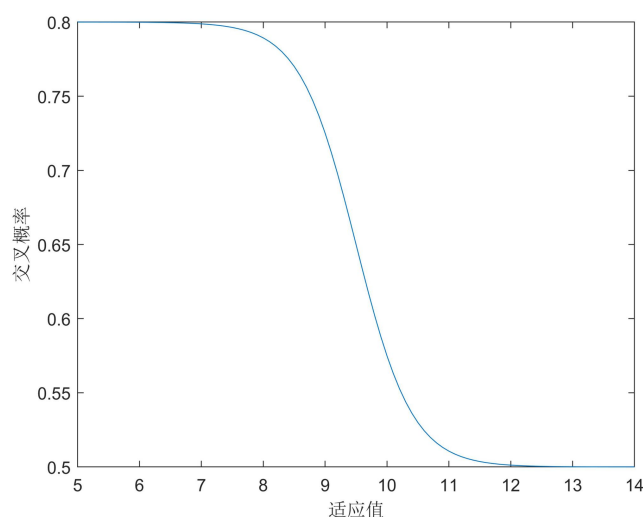


图 3-4 交叉概率函数图

3.5 模型求解

基于上述步骤，本文通过传统的遗传算法和改进遗传算法求解出在每次迭代过程中最优的适应度值，即居民满意度。算法改进前和改进后的每一代最优个体的适应度值随迭代次数的变化具体见图 3-5 和图 3-6。

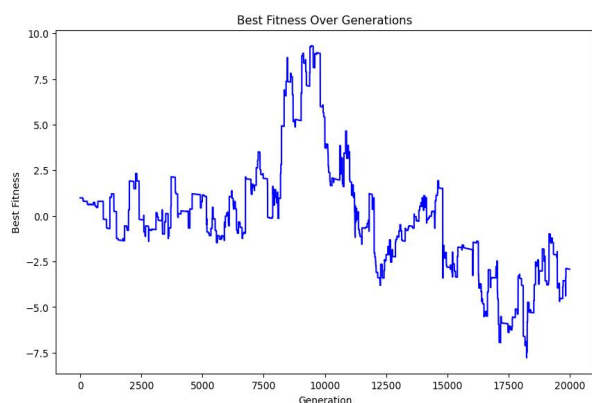


图 3-5 改进前种群适应度迭代图

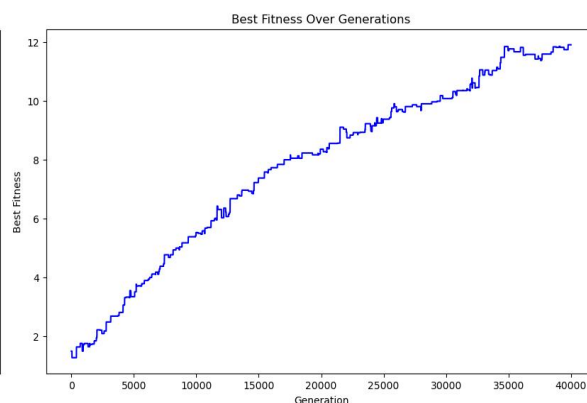


图 3-6 改进后种群适应度迭代图

从左图（改进前）可以看出，当我们随机生成初始解（即不干预初始解的生成）时，种群最优适应度值会出现低于 0 的情况，这主要是由于我们引入了惩罚因子，通过使用软性惩罚，根据违反约束的“严重程度”计算惩罚项。在迭代前期，图像出现震荡波动，种群适应度在一定范围内小幅度波动，优化方向不明显；在迭代中期，适应度值出现了显著提升，达到接近于的最优峰值，遗传算法效果明显；在迭代后期，适应度持续下降，可能是由于惩罚项会逐渐占据主导地位，违反约束越来越多。总体来说，通过引入惩罚因子并基于传统的遗传算法得出的效果并不是很好，虽然加快了收敛速度，但得到的解的质量较差，所以在此基础上对遗传算法进行了改进。

从右图（改进后）可以看出，当我们按照前文所述干预初始解的生成，并且舍弃软惩罚时，能获得比较好的结果。在迭代前期，可以明显看出适应度值在稳步上升，说明我们使用的选择、交叉、变异策略良好，个体质量逐步提升；在迭代中后期，适应度值仍在持续增长，但增幅会变缓，曲线中有小幅度的震荡，说明算法保持了一定的多样性；在迭代尾段，最优适应度值在 12 附近发生频繁波动，说明很可能已经达到最优解。

改进算法之后，我们能得到较优的搬迁方案。整体搬迁方案以表格的形式给出，包含每个居民是否搬迁、如果搬迁其满意度的取值等具体情况。总体来说，需要搬迁的居民的数量为 68 户，占比为 60.2%。在需要搬迁的居民中，搬迁后的地块面积增大的居民数量为 68，占总量的 100%，说明此搬迁方案中，搬迁的居民都能享受到更大的住房面积；搬迁后采光条件变好的居民数量为 21，占总量的 30.9%，说明在此搬迁方案中，有三成的搬迁居民能享受到更好的采光条件；能住进修缮过的房屋的居民数量为 47，占总量的 69.1%，说明在此搬迁方案中，有将近七成的搬迁居民能住搬入修缮过后的房屋。

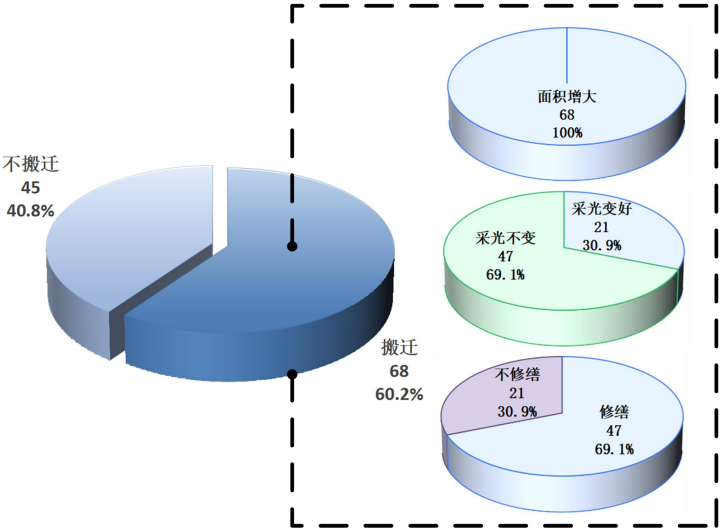


图 3-7 搬迁方案饼状图

在此我们选出了两个具有代表性的居民搬迁方案进行具体分析（见表 3），其他具体细节详见支撑材料附件 1。

表 3 部分用户搬迁方案

地块 ID	原始信息	搬迁信息	满意度	修缮情况
5	面积:68 m ² 采光:3	新地块:46	面积:0.079	修缮:是
		面积:75 m ² 采光:3	采光:0	金额:15 万元 满意度:0.075
41	面积:77 m ² 采光:1	新地块:42	面积:0.158	修缮:否
		面积:91 m ² 采光:2	采光:0.150	金额:0 满意度:0

对于地块 ID 为 5 的居民，需要搬迁至 ID 为 46 的地块，面积增量为 $7m^2$ ，搬迁前后的地块采光条件不变，因此开发商需要对搬迁后的地块进行修缮。该居民面积补偿满意度归一化后的取值为 0.079，采光补偿满意度为 0，修缮补偿满意度为 0.075。

对于地块 ID 为 41 的居民，需要搬迁至 ID 为 42 的地块，面积增量为 $14m^2$ ，采光条件增量为 1，因此开发商不需要对搬迁后的地块进行修缮。该居民面积补偿满意度归一化后的取值为 0.158，采光补偿满意度为 0.150，修缮补偿满意度为 0。

3.6 附加影响因素

除了前文所提及的 3 种因素，我们还考虑到以下 5 种因素可能也会影响到居民的搬迁意愿。我们站在住户的角度上考虑到原住户的偏好，并且考虑到数据可得性，进行了详细的定性以及定量的讨论，新增了以下 5 个主要因素。作为开发商，在实际的搬迁方案中，可以采用调查问卷法收集居民意愿。使用层次分析法对 8 个因素进行重要性排序，赋予权重，加权后作为搬迁意愿函数。

1.社区生活便利性。该因素包含三个一级指标：**是否临近街道衡量、是否就近安置衡量以及是否邻里共同搬迁。**临街区域通常优先享受基础设施改善和环境整治，居民认为其发展潜力大、居住价值高。与原先巷道深处、通行不便的住处相比，靠近街道的位置被视为居住条件的改善，更易被接受。**就近安置。**居民更愿接受就近安置^[8]，尤其是迁入地块靠近原院落时，心理抗拒降低。毗邻搬迁能保持原有生活圈，不打破邻里关系，避免社交网络割裂。熟悉的环境增强安全感与归属感，帮助居民更快适应新居。**邻里关系。**搬迁时若能与亲朋好友住一起，能减少适应成本，增强归属感和安全感。熟人互助有助于日常生活，提升搬迁接受度，并形成正向示范效应。

2.地块周边房屋密集程度。该因素包含两个二级指标：**院落密集程度和院落内住户密集程度。**密集环境会降低私密性，导致公共空间拥挤。高密度还可能带来垃圾堆积、噪音等问题，增加邻里摩擦，因此对高密度安置存在排斥。

3.6.1 层次分析法

层次分析法是将决策问题分层，通过定性定量结合分析，确定各因素权重以辅助决策的方法，广泛应用于多领域。为确定影响搬迁方案的各因素的重要性，量化评价各个搬迁方案，本文采用层次分析法进行计算。

(1) 建立结构层次模型

把决策问题分为 3 个层次，目标层 M 的目标是评价搬迁方案是否合理；准则层 C 的一级指标包括搬迁补偿 C_1 、社区生活便利性 C_2 和房屋密集程度 C_3 ；二级指标主要包括 C_{11} 、 C_{12} 、 C_{13} 、 C_{21} 、 C_{22} 、 C_{23} 、 C_{31} 、 C_{32} 。

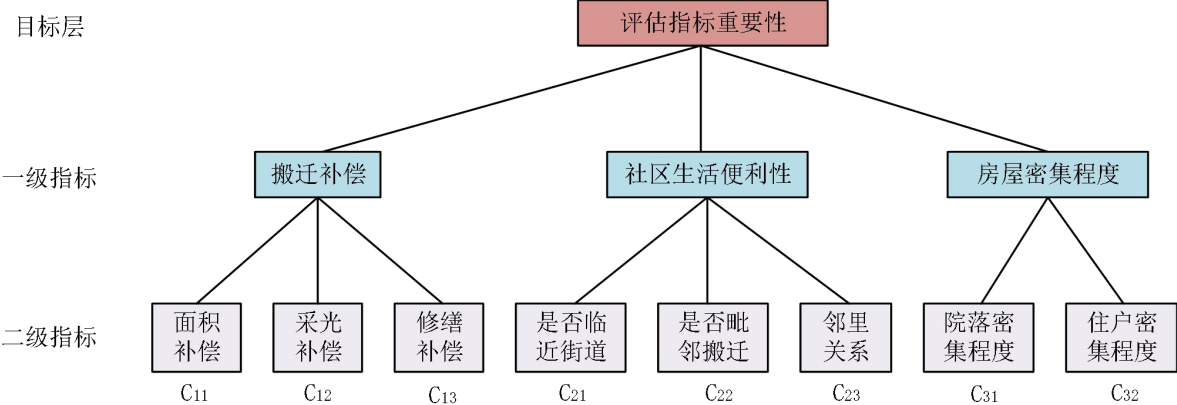


图 3-8 层次分析图

(2) 模型求解

对准则层指标进行两两打分构造判断矩阵，进行一致性检验后，得出各个指标的权重如下表：

表 4 准则层指标权重

准则层	权重
搬迁补偿	0.667
社区生活便利性	0.222
房屋密集程度	0.111

通过对方案层各判断矩阵进行一致性检验，所有 CR 值均小于 0.1，说明本模型判断逻辑合理，满足层次分析法的一致性要求，可以得出如下的总权重整合表：

表 5 决策层指标权重

决策层	全局权重
面积补偿	$0.667 \times 0.500 = 0.333$
采光补偿	$0.667 \times 0.296 = 0.197$
修缮补偿	$0.667 \times 0.204 = 0.136$
邻里关系	$0.222 \times 0.571 = 0.127$
是否临近街道	$0.222 \times 0.286 = 0.063$
是否毗邻搬迁	$0.222 \times 0.143 = 0.032$
住户密度	$0.111 \times 0.750 = 0.083$
院落密集程度	$0.111 \times 0.250 = 0.028$

3.6.2 模型建立

考虑到新地块与主街道的距离、住户的密集程度、是否保持原有邻里关系等因素也

会对居民的搬迁意愿产生影响，在本节中，我们在目标函数中增加了上述的 5 个因素，可以增加模型的适用性和解释性。

(1) 目标函数

$$\begin{aligned} \max z = & w_1 \sum_{i,j} x_{ij} \cdot IS'_{ij} + w_2 \sum_{i,j} x_{ij} \cdot IL'_{ij} + w_3 \sum_{i,j} x_{ij} \cdot RB'_{ij} + w_4 \sum_{i,j} x_{ij} \cdot AS'_{ij} \\ & + w_5 \sum_{i,j} x_{ij} \cdot YC'_{ij} + w_6 \sum_{i,j} x_{ij} \cdot PC'_{ij} + w_7 AR' + w_8 NR' \end{aligned} \quad (3-11)$$

各个符号的具体解释详见下表，符号带有上标代表对数据进行了标准化处理，各个因素的权重具体见表。

表 6 额外因素符号说明

符号	含义	说明
AS_i	变量	第 i 个地块所属的第 k 个院落是否临近街道，为 0-1 变量，认为如果某外围院落至少有一条临近街道，则取值为 1，否则为 0
$\sum_{i,j} AS_{ij}$	临近街道满意度指标	$\sum_{i,j} AS_{ij} = \sum_{i,j} (AS_j - AS_i)$
YC_i	变量	第 i 个地块所属的第 k 个院落密集程度，采用院落邻接图中第 k 个院落顶点的度衡量
$\sum_{i,j} YC_{ij}$	院落密集程度满意度指标	$\sum_{i,j} YC_{ij} = \sum_{i,j} (YC_i - YC_j)$
PC_i	变量	第 i 个地块所属的第 k 个院落内住户密集度，等于院内住户数量除以地块数量， $PC_i = \sum_{j \in \Omega_k} x_j / \Omega_k $ ， $ X $ 表示集合 X 的基数
$\sum_{i,j} PC_{ij}$	住户密集度满意度指标	$\sum_{i,j} PC_{ij} = \sum_{i,j} (PC_i - PC_j)$
AR_i	变量	是否毗邻搬迁，如果属于院落 k 的居民 i 搬到了属于毗邻的院落 l 的地块 j ，则取值为 1，否则为 0
AR	毗邻搬迁满意度指标	$AR_{ijk} = \begin{cases} 1, & \text{if } adj_{kl}=1, i \in V_k, j \in V_l \\ 0, & \text{else} \end{cases}$ $AR = \sum_i AR_i$
NR	邻里关系满意度指标	若两个（或多个）住户原来同属于一个院落，并搬迁到另外同一个院落，则 $NR = NR + 1$
$\sum_{i,j} IS_{ij}$	面积补偿满意度指标	$\sum_{i,j} IS_{ij} = \sum_{i,j} s_j - s_i$ 等于搬入地块的面积减去搬出地块的面积
$\sum_{i,j} IL_{ij}$	采光补偿满意度指标	$\sum_{i,j} IL_{ij} = \sum_{i,j} l_j - l_i$ 等于搬入地块的采光减去搬出地块的采光
RB_{ij}	修缮补偿满意度指标	$RB_{ij} = \sum_j r_j \cdot s_j \cdot rcost$ ，搬入地块的修缮补偿金额

(2) 约束条件

根据前文针对于问题一的分析，本节我们只调整了目标函数，约束条件同前文。

3.6.3 模型求解

根据层次分析法计算得到的各指标权重，以居民整体满意度为目标函数，沿用上文模型的约束条件，采用改进遗传算法进行求解，得出每一代最优个体的适应度值随迭代次数的变化，具体见图 3-9。

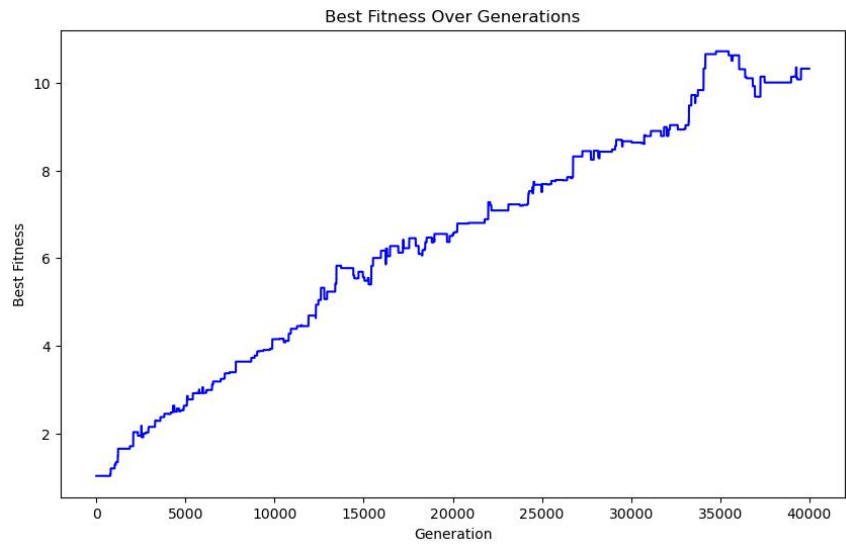


图 3-9 种群最优适应度迭代图

从图中可以看出，在迭代前期，算法基于选择、交叉和变异机制广泛探索搜索空间，使最佳适应度稳定上升，能有效避免陷入局部最优，能全面挖掘提升居民满意度的方案；在迭代中期，算法面对局部最优干扰，但变异操作带来的随机性使其不断尝试跳出局部最优解，持续向全局最优迈进；在迭代后期，算法收敛，最佳适应度值保持在相对稳定的状态，在 11 附近跳动，保障了解的可靠性，表明算法能够在有限的迭代次数内逼近或找到最优解。

与前文中仅考虑三个因素的目标函数不同，本文通过引入八个因素来增强模型的适用性。我们不仅评估了居民搬迁前后硬性指标（如面积补偿、采光补偿及修缮补偿）的变化，还考虑了软性指标（如新地块与主街道的距离、住户密集度等）的变化。整体搬迁方案以表格形式呈现，包含每个居民是否搬迁、搬迁后的满意度、院落密度变化等详细信息。具体结果显示，**需要搬迁的居民为 57 户，占总居民的 50.4%**，较仅考虑三个因素时减少。搬迁后，所有搬迁居民的住房面积均有所增加；其中 **20 名居民的采光条件有所改善**，占 35.1%；37 名居民将住进修缮过的房屋，占 64.9%。在搬迁前后，33 名居民的临街状态未变化，12 名居民从临街院落搬至不临街院落，另 12 名则反向搬迁。对于居民的住户密度，24 名居民搬至密度较低的地块，住户满意度提高，29 名搬至密

度较高的地块，满意度降低；搬迁后，14 名居民搬到院落密度较高的院落，24 名居民搬到密度较低的院落。仅有一户居民能够实现毗邻搬迁，绝大多数居民未能搬至毗邻院落。具体细节详见**支撑材料附件 2**。

由上述分析可知，需要考虑五个额外影响居民搬迁意愿的因素时，搬迁方案的评价维度增多，导致居民对迁入地块的满意度评估更加严格，符合搬迁条件的地块数量减少，从而使最终愿意搬迁的居民数量相较于三因素时有所下降。对于开发商来说，其可以通过给每个指标进行线性赋权，站在居民的角度上进行建模求解，从而在得出的最优搬迁方案的基础上规划并落实搬迁工作。

4.问题二模型建立与求解

4.1 问题二分析

针对问题二，按照俞宁^[9]等人的方法，我们首先构建了一个多目标优化模型，综合考虑三个核心目标：一是最大化居民的搬迁满意度，确保搬迁过程符合居民的居住意愿与补偿期望；二是最大化腾退整院的总面积，并尽可能使空置院落在空间上实现毗邻配置，以提升后续开发的连续性和整体价值；三是最小化参与搬迁的住户数量，以降低搬迁沟通成本与资源投入。问题一中的模型采用了线性加权的方式整合成单目标优化问题，但由于问题二需要我们同时优化多个相互冲突的目标，为在三个目标函数中权衡搬迁方案，我们选用非支配排序遗传算法（Non-dominated Sorting Genetic Algorithm II，NSGA-II），获得一组近似 Pareto 最优解集。最后，我们将使用得到的 Pareto 最优解集计算成本、租金收入以及最终整院面积整院面积，从而计算最终盈利。

4.2 模型建立

4.2.1 基础概念定义

在构建问题二的模型之前，我们需要定义以下概念：

1.空置整院判定逻辑

某一院落是否构成“空置整院”，取决于该院落内所有原有居民地块是否已全部搬迁，或该院落自始即无居民居住。 V_k 表示院落 k 中搬迁之前有居民的地块集合，如果居民从地块 i 中搬出，那么有 $\sum x_{ij}=1$ ，我们采用 0-1 变量 $z_k \in \{0,1\}$ 来表示院落 k 是否被完全腾空，定义如下：

$$z_k = \begin{cases} 1, & \text{if } \forall i \in V_k, \sum_j x_{ij} = 1 \\ 0, & \text{else} \end{cases} \quad (4-1)$$

2.连通整院（毗邻院落） C_m 的定义与识别

通过上述方法识别出全部的空置院落，我们将空置整院之间的邻接关系采用 $adj_{kl} \in \{0,1\}$ 进行表示，如果院落 k 与院落 l 毗邻，则 $adj_{kl}=1$ ，从而构建空置整院邻接图。

连通整院块（毗邻组） C_m 的识别：我们需要识别若干连通的空置整院毗邻组，记为： $C_1, C_2, \dots, C_m$ ，每个 C_m 是一个所有整院的集合，其中每个院落 $k \in C_m$ 都满足 $z_k=1$ ，且任意两个院落在该集合中彼此连通。

3.收益增益因子定义

根据整院效益，完整的空置杂院出租/开发价值更高，为 30 元/平米/天。根据毗邻效益，若空置整院彼此毗邻，则出租价值增加为原来的 1.2 倍。因此，我们定义面积增益因子为：

$$\alpha_m = \begin{cases} 1, & |C_m| = 1 \\ 1.2, & |C_m| \geq 2 \end{cases} \quad (4-2)$$

其中， $|C_m|$ 表示毗邻组 C_m 中的空缺整院数量

上式表明在图中作为孤立点的完整空置杂院面积增益因子为 1，若空置整院彼此毗邻（对应图中多点联通），则收益增益因子为 1.2。

4.2.2 目标函数

目标函数 1：使得居民的满意度最大，在本节我们沿用问题 1 的居民满意度函数，并且采用问题 1 求得的权重。具体见式 4-3：

$$\max z = w_1 \sum_{i,j} x_{ij} \cdot IS'_{ij} + w_2 \sum_{i,j} x_{ij} \cdot IL'_{ij} + w_3 \sum_{i,j} x_{ij} \cdot RB'_{ij} \quad (4-3)$$

目标函数 2：最小化搬迁住户的数量，具体见式 4-4。

$$\min P = \sum_i \sum_j x_{ij} \quad (4-4)$$

目标函数 3：为了使完整院落尽可能相互毗邻，且使得总面积最大，考虑到整院效应和毗邻效应，我们将该目标表示为使用收益增益因子加权后的单位面积收益。具体见式 4-5。

$$\max R = 30 \cdot \sum_m [\alpha_m \cdot A(C_m)] \quad (4-5)$$

其中， $A(C_m)$ 表示毗邻组所包含的所有空置整院面积之和。

4.3 NSGA-II 算法

NSGA-II 算法是迄今为止最流行的元启发式算法，已被很多研究者用于解决多目标优化问题^[10]，并且它通常是处理现实世界多目标优化问题的首选求解器^[11]。NSGA-II 算法的主要机制包括：

1.非支配排序

将种群按支配关系进行分层：若一个个体不被任何其他个体在所有目标上同时优

于，则称其为非支配解，归为第一层；去除该层后，重复该过程，依次构建第二层、第三层等，形成多个等级。

2.精英策略

在每代演化中保留当前种群中的优质解，防止优秀个体被淘汰，从而提升算法的稳定性与收敛效率。

3.拥挤度距离

衡量个体在目标空间的局部密度。较大的拥挤度表示该个体位于解集边缘或稀疏区域，有助于维持解的多样性与帕累托前沿的均匀分布。

4.快速非支配排序+拥挤度比较

综合考虑排序等级和局部密度，作为个体选择的优先依据，确保解的优良性与多样性兼顾，是 NSGA-II 核心的选择机制。

我们采用稀疏初始化策略对搜索空间进行粗化，加快算法的收敛速度^[12]。

4.4 搬迁方案结果计算

4.4.1 利益计算函数定义

1.成本计算函数

成本=沟通成本+修缮成本+搬迁损失成本：

$$Cost = 3 \cdot \sum_{i,j} x_{ij} + \sum_j r_j \cdot s_j \cdot rcost + \sum_{i,j} x_{ij} \cdot s_j [8 \cdot (1 - o_j) + 15 \cdot o_j] \cdot 10^{-4} \cdot T \quad (4-6)$$

式中， T 表示搬迁导致不能收取租金的 4 个月，单位为天，即 $T = 120$ ， $3 \cdot \sum x_{ij}$ 表示沟通成本， $\sum r_j \cdot s_j \cdot rcost$ 表示修缮成本。

搬迁损失成本=搬入地块的面积×目标地块租金单价×时间：

$$4 \text{ 个月搬迁损失成本} = \sum_{i,j} x_{ij} \cdot s_j \cdot [8 \cdot (1 - o_j) + 15 \cdot o_j] \cdot 10^{-4} \cdot T \quad (4-7)$$

2.最终整院面积计算函数

$$Area = \sum_m A(C_m) \quad (4-8)$$

3.租金收入计算函数

假设一个承包年限 Y ，在承包年限内讨论，我们假设承包年限为 10 年，取 $Y = 10$ 。

最终租金收入之和=整院面积收入+散户面积收入，即：

$$\begin{aligned}
Income = & \sum_{i,j} [(1-x_{ij}) \cdot (1-t_j) \cdot s_j \cdot [8 \cdot (1-o_j) + 15 \cdot o_j] \cdot 10^{-4} \cdot Y \cdot 365] \\
& + \sum_{i,j} [x_{ij} \cdot t_i \cdot s_i \cdot [8 \cdot (1-o_j) + 15 \cdot o_j] \cdot 10^{-4} \cdot (Y \cdot 365 - T)] \\
& + 30 \cdot \sum_m [\alpha_m \cdot A(C_m)] \cdot T \cdot 10^{-4} \\
& + 30 \cdot \sum_m [\alpha_m \cdot A(C'_m)] \cdot (Y \cdot 365 - T) \cdot 10^{-4}
\end{aligned} \tag{4-9}$$

式中，公式第一个部分 $(1-x_{ij}) \cdot (1-t_j)$ 表示地块 j 没有人居住，居民 i 不搬入地块 j ，计算该部分面积 0 至 10 年的收益；公式第二个部分 $x_{ij} \cdot t_i$ 表示地块 i 有人居住，且原居民搬出，该部分面积只计算 4 个月至 10 年的收益。 s_j 表示地块 j 的面积， o_j 表示地块的朝向；公式后半部分 $30 \cdot \sum [\alpha_m \cdot A(C_m)] \cdot T \cdot 10^{-4} + 30 \cdot \sum [\alpha_m \cdot A(C'_m)] \cdot (Y \cdot 365 - T) \cdot 10^{-4}$ 表示完整院落的租金收益， $A(C_m)$ 表示搬迁前的空置毗邻整院块的面积， $A(C'_m)$ 表示搬迁后的空置毗邻整院块的面积。

4.最终盈利计算函数

最终盈利=收入-成本（单位：万元），即：

$$Profit = Income - Cost \tag{4-10}$$

4.4.2 帕累托解集

根据前文的具体分析，我们设定多个目标，通过采用 NSGA-II 算法进行求解，得到帕累托解集，种群个体数为 50，其中 49 个点为帕累托前沿点，1 个解被其他点支配，具体见图 4-1 和图 4-2。

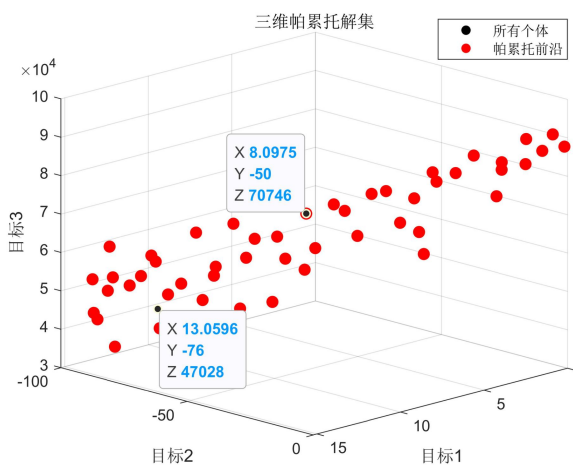


图 4-1 三维帕累托解集图

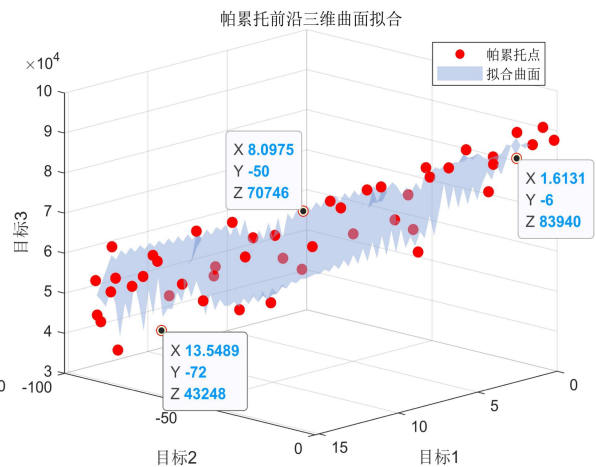


图 4-2 帕累托前沿曲面拟合图

两张图均展示了在使居民满意度最大（目标 1）、最小化搬迁住户的数量（目标 2）

及使单位面积收益最大（目标 3）这三个目标下的帕累托前沿情况。红色散点在三维空间中的分布反映了不同目标组合下的最优解集合。

4.4.3 结果呈现

通过采用 NSGA-II 算法进行求解，我们得到了包含 49 个解的帕累托前沿解集（详见附件 3），在附件 3 中我们给出了这 49 个解具体的搬迁方案，包括哪些院子的居民进行了搬迁、搬迁前后的地块 ID 及搬迁之后整院的院落 ID。我们在附件 4 中给出了这 49 个搬迁方案对应的搬迁成本，主要包括沟通成本、修缮成本以及搬迁损失成本，**得出最大的搬迁成本是 1794.448 万元，不超过 2000 万元**，所以对于我们得到的帕累托前沿中的方案，都无需追加备用资金。我们还在附件 5 中给出了开发商针对于这 49 个方案的盈利，且**最大盈利为 375214.3 万元**。在附件 6 中，我们给出了各个方案的整院面积，**最大整院面积是 20002 平方米**。在附件 7 我们给出了各个搬迁方案对应的三个目标函数值，通过分析可知这三个目标函数之间存在此消彼长的关系。

5.问题三模型建立与求解

5.1 问题三分析

问题三需要找到是否存在一个搬迁“拐点”，在该拐点之后，继续搬迁虽然能提高收益，但单位成本对应的收益提升变小，即性价比 m 下降。如果存在拐点，输出拐点对应的搬迁方案和收益成本结构；如不存在，讨论在 m 降到多少时才有拐点。首先，需要构建性价比计算函数，其次，基于问题 2 得到的帕累托解集，计算帕累托最优解的性价比。然后，将性价比作为目标函数，取出性价比高于 20 的可行解，将其作为初始种群中的个体，追踪并记录其目标函数值在每一次变异后的改变，如果经过某个变化（搬迁操作）后，性价比开始下降，那么将该操作视为性价比变低的拐点。最后，计算共投入多少成本，最终的整院面积和最终的收入及盈利。

5.2 性价比计算函数构建

首先，我们构建性价比计算函数如下：

$$m = \frac{Income - Income(0)}{3 \cdot \sum_{i,j} x_{ij} + \sum_j r_j \cdot s_j \cdot rcost} \quad (5-1)$$

式中， Y 为开放商承包年限，在问题三中 $Y=10$ ； $Income$ 由上文公式 4-10 计算得到； $Income(0)$ 表示不搬迁的面积收益，即搬迁方案决策变量 $x_{ij}=0, \forall i, j$ 。该公式表示性价比=搬迁十年的租金收益相比于不搬迁的增量/实际搬迁投入。搬迁投入相比搬迁成本不包含搬迁损失成本。

5.3 结果分析

基于前文模型和算法，迭代得到 16 个性价比大于 0 的解集，绘制柱状图如下：

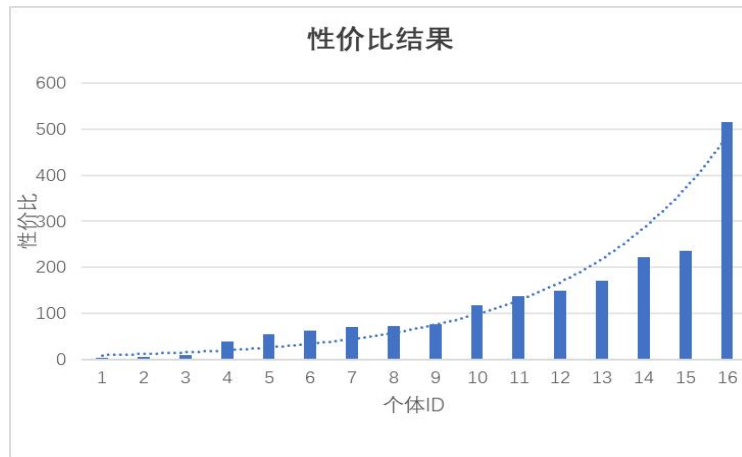


图 5-1 性价比柱状图

由图 5-1，我们可以观察得到，性价比拐点出现在个体 10 和个体 11 附近，对应附件 10 中个体 ID 为 12 和 16 的搬迁方案。个体 ID 为 12 的搬迁方案对应地块 ID 由 14 搬到 143，120 搬到 82，141 搬到 316，235 搬到 149，其余住户不搬迁，结算成本为 64.82 万元（具体见附件 9），整院面积为 20127 平方米，最终盈利为 373986 万元；个体 ID 为 16 的搬迁方案对应 181 搬到 480，215 搬到 436，459 搬到 308，其余住户不搬迁，结算成本为 40.56 万元，最终盈利为 370714 万元，整院面积为 19878 平方米。

6.搬迁决策软件框架设计

6.1 问题四分析

问题四的重点在于将前期建立的数学模型转化为一套具备推广价值的应用系统，用于支持更广泛的老旧街区改造工作。本问题需要我们基于上文的理论建模和算法设计开发一个功能完整、操作便捷的搬迁辅助决策平台，实现地块与居民数据的自动读取，基于满意度评估和多目标优化逻辑，生成多种可选搬迁方案，并提供如整院腾退面积、搬迁户数、经济支出等核心指标的量化输出。系统还应具备参数可调、方案对比与结果可视化等功能，为管理部门在不同区域和政策背景下制定合理搬迁方案提供可靠支撑。

6.2 软件设计与实现

为实现对“老城区平移置换搬迁”问题的智能化辅助决策，提升模型的推广适应性与工程可操作性，我们基于前述模型构建了一套具有通用性的软件框架。该系统集成了数据处理、满意度建模、多目标优化与可视化输出等模块，能够针对不同街区结构和搬迁规划参数，自动生成最优搬迁方案及配套评估报告，具备实际价值。

6.2.1 系统功能架构

整体软件架构包括输入—建模—优化—输出四层结构，建模部分包括数据预处理模块和居民满意度评价模块，优化部分包括优化算法模块。系统的功能架构见图 6-1：

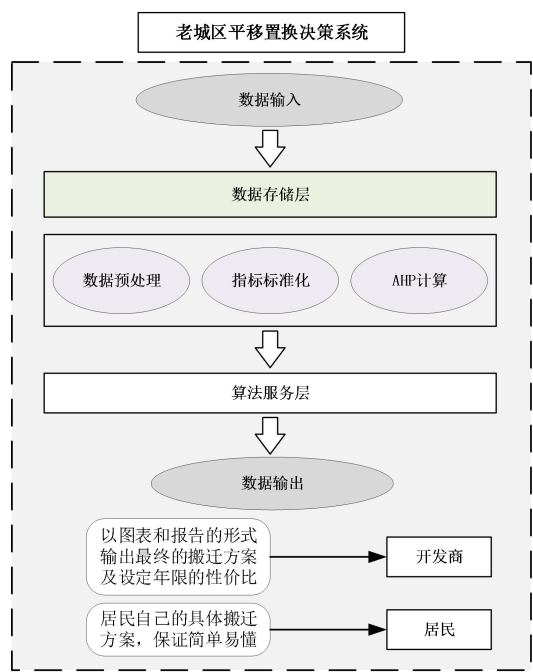


图 6-1 系统功能架构图

6.2.2 模块功能设计

1.数据输入模块

用户既能够借助图形化界面，以直观操作的方式输入原始数据，也可以通过导入 Excel 文件的形式来完成原始数据的输入。需要用户人工输入的参数主要包括居民的基本信息、地块信息等，详见图 6-2：

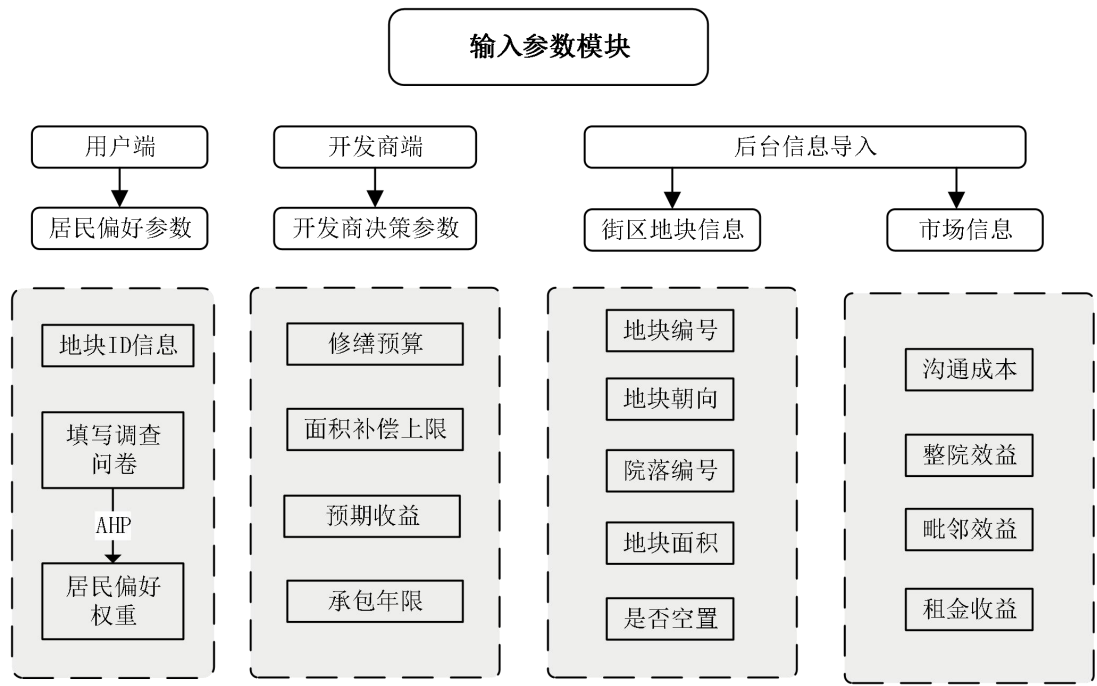


图 6-2 输入参数

2.数据预处理模块

根据输入的相关数据，需要对一些数据进行预处理操作，具体包括下述操作：地块解析，解析地块与院落 ID、朝向、面积和居住状态；邻接矩阵构建，生成院落之间的毗邻关系矩阵。

3.满意度评价模块

在满意度评价模块，主要包括下述部分：指标标准化，使用最大-最小归一化对面积补偿、采光补偿以及修缮补偿进行标准化处理；AHP 计算，加载用户设定的判断矩阵，计算补偿权重并输出满意度矩阵。

4.优化求解模块

在优化与求解模块，我们主要采用问题二和问题三的模型进行优化求解，通过在算法中使用基于实数的搬迁方案编码，并且结合可行约束，同时优化多个目标，输出一组帕累托最优解，并计算各个解的在承包年限内的性价比，供政策制定者灵活选用。

5.结果输出与可视化模块

经过上述操作，该系统会以图表和报告的形式输出最终的搬迁方案。对于开发商来说，主要可以看到以下具体信息：搬迁对照表（每位居民迁入迁出信息）；腾退院落的清单及总面积；居民对方案的满意度、开发商搬迁成本与效益统计；居民搬迁路径与院落分布的可视化图；PDF 及 Excel 报告。对于居民来说，只能通过输入自己的地块 ID 来查看对应的方案，结果直观，简单易懂。

7.模型评价与推广

7.1 模型评价

7.1.1 模型优点

1.考虑全面：模型在设计上充分融合了居民的补偿诉求、开发商的收益成本以及实际操作中的多项约束，能够较为真实地反映老旧街区改造与搬迁过程中的多方博弈，为实现利益平衡提供了有力的分析工具。

2.方法科学：在建模方法上，采用层次分析法对影响因素进行定量赋权，并结合遗传算法与非支配排序遗传算法（NSGA-II）进行求解，有效应对了多目标、多约束的Np-hard问题，提升了模型的科学性与可靠性。

3.具备实用性：模型参数可灵活调整，能够适应不同城市、不同地块条件下的搬迁需求，具备较强的现实适配能力，可为城市更新中的实际搬迁规划提供具有可操作性的决策支持。

7.1.2 模型不足

1.假设存在理想化：模型中部分假设可能与实际存在偏差，例如假定“如果房屋进行了修缮翻新，居民就会同意搬迁”，而现实中居民意愿往往受多种社会心理和历史因素影响。

2.对数据质量较为敏感：模型运行结果高度依赖于输入数据的准确性与完整性，若数据存在缺失或误差，将直接影响优化结果的合理性，因而前期数据收集工作量较大。

3.求解复杂度较高：在面对大规模地块与居民数据时，模型计算量大，优化算法在精度与效率之间可能存在一定权衡，对硬件环境与算法调优提出了更高要求。

7.2 模型推广

1.适用于不同城市：本模型在构建逻辑与求解方法上具有较强的通用性，适用于不同城市在老旧街区更新过程中面临的共性问题。尽管各地具体情况存在差异，但通过对模型参数、补偿机制及约束条件的调整，可使其灵活适配于不同区域的改造实际，为各地政府和开发单位提供系统化的搬迁决策参考。

2.可拓展至其他搬迁场景：除老旧街区外，模型亦可推广至城中村整治、棚户区改造等其他类型的搬迁与重建场景。上述情形同样面临人口安置、资源分配、利益平衡等

核心问题，本模型所提供的优化思路与方法可为相关问题的系统化解决提供理论依据与技术路径。

3.与地理信息系统融合应用：结合地理信息系统（GIS）技术，可实现搬迁方案空间分布的可视化展示，辅助评估搬迁行为对城市空间结构、交通组织等方面的影响，为规划部门提供更加直观、全面的参考依据。

参考文献

- [1] 王承华, 李智伟. 城市更新背景下的老旧小区更新改造实践与探索——以昆山市中华北村更新改造为例[J]. 现代城市研究, 2019(11): 104-112.
- [2] 曲帅, 王逸初, 邵禹涵. 我国城市更新的实践探索、潜在挑战与实现路径[J]. 价格理论与实践, 2025: 1-4.
- [3] 孙威, 王晓楠, 盛科荣. 基于文献计量方法的国内外城市更新比较研究[J]. 地理科学, 2020, 40(8): 1300-1309.
- [4] 俞宁, 冯鑫, 汤爱华, 等. 基于改进 NSGA-II 算法的电动汽车充电站选址方法[J]. 郑州大学学报（理学版）, 2024: 1-6.
- [5] 杨世忠, 孙崇国, 李善伟. 基于改进 GA 的变风量空调系统优化控制仿真[J]. 计算机仿真, 2021, 38(11): 230-234, 253.
- [6] 湛文静, 李泳科. 基于改进遗传算法的路径规划问题相关研究综述[J]. 计算机与数字工程, 2023, 51(7): 1544-1550.
- [7] 许新华, 高刚, 徐永嘉. 基于改进遗传算法的综合能源优化调度方法[J]. 电气开关, 2023, 61(5): 35-38, 42.
- [8] Niu Q, Xi Y, Yushi, Hu Z, Zhouwei, et al. After forced relocation: assessing the impact of urban regeneration on residents' living spaces in wuhan, china[J]. Housing Policy Debate: 1-19.
- [9] 俞宁, 冯鑫, 汤爱华, 等. 基于改进 NSGA-II 算法的电动汽车充电站选址方法[J]. 郑州大学学报（理学版）, 2024: 1-6.
- [10] Nebro A J, Galeano-Brajones J, Luna F, et al. Is NSGA-II ready for large-scale multi-objective optimization?[J]. Mathematical and Computational Applications, 2022, 27(6): 103.
- [11] Fang W, Guan Z, Su P, et al. Multi-objective material logistics planning with discrete split deliveries using a hybrid NSGA-II algorithm[J]. Mathematics, 2022, 10(16): 2871.
- [12] Li M, Ma H, Lv S, et al. Enhanced NSGA-II-based feature selection method for high-dimensional classification[J]. Information Sciences, 2024, 663: 120269.

附录

本文基于 Python 3.10 软件和 Matlab R 2019a 进行建模，相关代码如下：

问题 1：三因素作为目标函数的遗传算法

```
# %%  
import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd  
import random  
import copy  
  
# %%  
# 参数设置  
DNA_SIZE = 113 # DNA 大小 1-9 表示搬迁前 i 10-18 表示搬入 j 19 表示 xij 的值 20 表示 rj  
POP_SIZE = 50 # 种群大小  
CROSSOVER_RATE = 0.8 # 交叉率  
MUTATION_RATE = 0.2 # 变异率  
N_GENERATIONS = 40000 # 代数  
rcost = 0.2 # 单位面积翻新成本，单位为万  
Time = 120 # 搬迁时间 天  
w1 = 0.1 # 金钱的权重，包括 沟通成本、修缮成本、搬迁时间内租金成本  
w2 = 0.9 # 面积损失的成本  
num_residents=113  
num_empty_houses=371  
M = 99999 # 极大常数  
  
# %%  
# 读取地块信息数据  
file_path = '附件一：老城街区地块信息.xlsx'  
data = pd.read_excel(file_path)  
# 提取相关数据  
areas = data['地块面积'].values # 地块面积  
orientations = data['地块方位'].apply(lambda x: 1 if x in ['南', '北'] else 0).values # 0 为东西，1 为南北  
has_residents = data['是否有住户'].values # 是否有住户  
light = data['地块方位'].apply(lambda x: 1 if x == '西' else (2 if x == '东' else 3)).values  
yards = data['院落 ID'].values  
dkIDs = data['地块 ID'].values  
  
areas_i = [] # 需要搬迁的用户的房屋面积  
areas_j = [] # 空房 房屋 面积  
orientations_i = [] # 需要搬迁 房屋 朝向  
orientations_j = [] # 空房 房屋 朝向  
light_i = [] # 需要搬迁 房屋 采光  
light_j = [] # 空房 房屋 采光
```

```

yard_i = [] #需要搬迁所属院落 ID
yard_j = [] #空房 院落 ID
dkID_i = [] #需要搬迁 房屋 地块 id
dkID_j = [] #空房 地块 id
for index in range(len(has_residents)):
    if has_residents[index]==1:
        areas_i.append(areas[index])
        orientations_i.append(orientations[index])
        light_i.append(light[index])
        yard_i.append(yards[index])
        dkID_i.append(dkIDs[index])
    elif has_residents[index] == 0:
        areas_j.append(areas[index])
        orientations_j.append(orientations[index])
        light_j.append(light[index])
        yard_j.append(yards[index])
        dkID_j.append(dkIDs[index])
    else:
        print("错误!!!!")

# %%
#读取毗邻矩阵
close_yard_data_df = pd.read_excel('毗邻矩阵.xlsx')
close_yard_data = close_yard_data_df.values.tolist()
#院落数据
yard_data_df = pd.read_excel('院落信息.xlsx')#有 5 列数据: 包含地块 ID、地块个数、原始居住状态(住
了几户人)、是否临街、顶点的度(相邻院落数量)
yard_data = yard_data_df.values.tolist()

# %%
#创建每一个住户搬迁的可行解
res_move_all_possibility = [[0] for _ in range(num_residents)]
for rs in range(len(areas_i)):
    for avil_targ in range(len(areas_j)):
        if areas_j[avil_targ] > areas_i[rs]:
            if 1.3*areas_i[rs] > areas_j[avil_targ]:
                if light_j[avil_targ] >= light_i[rs]:
                    res_move_all_possibility[rs].append(avil_targ+1)

# res_move_all_possibility_df = pd.DataFrame(res_move_all_possibility)
# res_move_all_possibility_df.to_excel("res_move_all_possibility.xlsx")

# %%

```

```

def Constraints_more(lst):
    # print(lst)
    # 只约束不重复的条件
    seen = set()
    for item in lst:
        if item != 0: # 忽略 0, 因为 0 表示不搬迁
            if item in seen: # 如果元素已经在集合中, 说明重复
                return False
            seen.add(item) # 将非零元素加入集合
    return True # 如果遍历完没有重复, 返回 True

def Constraints_s(lst):
    # 约束面积
    for n in range(num_residents):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 检查面积是否满足条件: areas_i[n] <= areas_j[lst[n]-1] 且 1.3 * areas_j[lst[n]-1] <=
areas_i[n]
            if areas_i[n] > areas_j[lst[n]-1] or areas_j[lst[n]-1] > 1.3 * areas_i[n]:
                # print(areas_i[n], 1.3 * areas_i[n], areas_j[lst[n]-1])
                return False
    return True # 如果遍历完没有问题, 返回 True

def Constraints_light(lst):
    # 约束采光
    for n in range(len(lst)):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 检查采光是否满足条件: light_j[lst[n]-1] >= light_i[n]
            if light_j[lst[n]-1] < light_i[n]:
                # print("新采光"+str(light_j[lst[n]-1]))
                # print("原采光"+str(light_i[n]))
                return False
    return True # 如果遍历完没有问题, 返回 True

def Constraints_rebuilt_money(lst):
    # 约束采光
    for n in range(len(lst)):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 判断重修花的钱是否超支, 首先要判断是否需要重修, 如果需要的话, 才判断花的钱
超支
            if areas_j[lst[n]-1] == areas_i[n] or light_j[lst[n]-1] == light_i[n]:
                rebuilt_money = rcost * areas_j[lst[n]-1]
                if rebuilt_money > 20:
                    return False
    return True

```

```

# %%
def translateDNA(pop):
    #解码
    relocation_plan = {}
    for num in range(num_residents):
        target_house = pop[num] # 从当前个体（kkk）中获取每个居民的目标房子
        if target_house != 0: # 如果该居民搬迁到某个房子
            # 判断是否需要修缮
            rjj = 1
            if areas_j[target_house-1] > areas_i[num] and orientations_j[target_house-1] >
orientations_i[num]:
                rjj = 0
            relocation_plan[num] = {
                "i": num+1, # 居民编号（从 1 开始）
                "j": target_house,
                "si": areas_i[num], # 假设房子编号从 1 开始，取相应的面积数据
                "sj": areas_j[target_house-1], # 目标面积数据
                "oi": orientations_i[num], # 同理取相应的朝向数据
                "oj": orientations_j[target_house-1], # 获取目标朝向
                "xij": 1,
                "rj": rjj # 根据面积、朝向等信息决定是否需要修缮
            }
        else:
            # 等于 0 的时候，就是不搬的情况
            relocation_plan[num] = {
                "i": num+1, # 居民编号（从 1 开始）
                "j": 999,
                "si": areas_i[num], # 假设房子编号从 1 开始，取相应的面积数据
                "sj": 999, # 目标面积数据
                "oi": orientations_i[num], # 同理取相应的朝向数据
                "oj": 999, # 获取目标朝向
                "xij": 0,
                "rj": 0 # 这里可以根据面积、朝向等信息决定是否需要修缮
            }
    return relocation_plan

# %%
def F(tl_data):
    # 目标函数，包括新增住房面积 0.6、采光补偿 0.1、修缮补偿 0.3
    z = 0
    for num in range(num_residents):
        if tl_data[num]["xij"] == 1: # 如果搬迁了
            z

```

```

+=0.6*(tl_data[num]["sj"]-tl_data[num]["si"])/53+0.1*rcost*tl_data[num]["sj"]*tl_data[num]["rj"]/20 +
0.3*(light_j[tl_data[num]["j"]-1]-light_i[num])/2
    return z

# %%
def get_fitness(pop_1):
    # 计算适应度
    tl_data = translateDNA(pop_1) # 获取 x, y 值
    # 计算目标函数，包括适应度和约束惩罚
    pred = F(tl_data) # 传入当前代数和最大代数，计算适应度（包括惩罚）
    return pred

# %%
def get_true_fitness(pop_12):
    # 计算适应度
    tl_data = translateDNA(pop_12) # 获取 x, y 值
    # 计算目标函数，包括适应度和约束惩罚
    pred = F(tl_data) # 传入当前代数和最大代数，计算适应度（包括惩罚）
    return pred

# %%
def select(pop, fitness, tournament_size=4):
    # 随机选择 `tournament_size` 个个体进行锦标赛
    selected_indices = random.sample(range(len(pop)), tournament_size)
    #selected_indices = [1,2,3,4]

    # 选择适应度最好的个体
    select_fitness = [fitness[p] for p in selected_indices]
    select_index = fitness.index(min(select_fitness))
    # best_index = selected_indices[0]
    # for index in selected_indices[1:]:
    #     if fitness[index] > fitness[best_index]:
    #         best_index = index

    # selected=pop[best_index]

    return pop[select_index],select_index

# %%
def crossover(pop_4, parent1, parent2, crossover_rate):
    pop_5 = copy.deepcopy(pop_4) # 创建一个种群的副本

```



```

if random.random() > crossover_rate:
    # 如果不进行交叉，直接返回父母
    return pop_5

# 随机选择两个交叉点
crossover_points = sorted(random.sample(range(1, DNA_SIZE), 2)) # 随机选择两个交叉点
crossover_point1, crossover_point2 = crossover_points

# 获取父母的交叉点基因值
org_parent1_point1_value = pop_4[parent1][crossover_point1]
org_parent1_point2_value = pop_4[parent1][crossover_point2]
org_parent2_point1_value = pop_4[parent2][crossover_point1]
org_parent2_point2_value = pop_4[parent2][crossover_point2]

# 交换父母基因
pop_5[parent1][crossover_point1] = org_parent2_point1_value
pop_5[parent1][crossover_point2] = org_parent2_point2_value
pop_5[parent2][crossover_point1] = org_parent1_point1_value
pop_5[parent2][crossover_point2] = org_parent1_point2_value
if Constraints_more(pop_5[parent1]) and Constraints_s(pop_5[parent1]) and
Constraints_light(pop_5[parent1]) and Constraints_rebuilt_money(pop_5[parent1]) and
Constraints_more(pop_5[parent2]) and Constraints_s(pop_5[parent2]) and
Constraints_light(pop_5[parent2]) and Constraints_rebuilt_money(pop_5[parent2]):
    return pop_5
else:
    return pop_4

# %%
def mutation(pop_7, parent1_num, mutation_rate):
    pop_8 = copy.deepcopy(pop_7) # 创建副本，避免直接修改原种群

    # 如果随机数小于变异率，则不进行变异，直接返回
    if random.random() > mutation_rate:
        return pop_7

    # 进行变异：选择一个随机的基因位置
    mutation_point = random.randint(0, len(pop_8[0]) - 1)

    # 变异为一个随机的地块编号
    # pop_8[parent1_num][mutation_point] = random.randint(0, 371)
    pop_8[parent1_num][mutation_point] =
random.sample(res_move_all_possibility[mutation_point], 1)[0]

    # 如果不满足约束条件，重新变异

```

```

        if Constraints_more(pop_8[parent1_num]) and Constraints_s(pop_8[parent1_num]) and
Constraints_light(pop_8[parent1_num]) and Constraints_rebult_money(pop_8[parent1_num]):
            return pop_8
        else:
            return pop_7

# %%
# 在 crossover_and_mutation 函数中, 确保为 select 传递 fitness 参数
def crossover_and_mutation(pop_3,parent1_num, crossover_rate, mutation_rate, fitness):
    parent1 = parent1_num
    parent2 = random.randint(0,49)
    #交叉操作
    pop_6= crossover(pop_3, parent1, parent2, crossover_rate)
    #变异操作
    pop_end = mutation(pop_6,parent1,mutation_rate)
    return pop_end

# %%
# pop = []
# for p in range(POP_SIZE):
#     # 确保至少有 80 个 0, 最多 113 个 0
#     num_zeros = random.randint(100, 110) # 随机选择生成多少个 0, 范围在 80 到 113 之间
#     num_non_zeros = 113 - num_zeros # 计算剩余的非零数字数量
#     # 生成不重复的其他数字 (从 1 到 371) 共生成 num_non_zeros 个
#     non_zero_values = random.sample(range(1, 372), num_non_zeros)
#     # 使用 random.choices 插入重复的 0
#     zeros = random.choices([0], k=num_zeros) # 生成 num_zeros 个 0
#     # 合并不重复的数字和 0
#     t_p = non_zero_values + zeros
#     # 打乱列表, 确保 0 不固定在前或后
#     random.shuffle(t_p)

#     # 使用 not 来确保在约束条件不满足时重新生成个体
#     attempts = 0
#     max_attempts = 1000000 # 设置最大尝试次数以防止死循环
#     while not (Constraints_more(t_p) and Constraints_s(t_p) and Constraints_light(t_p) and
Constraints_rebult_money(t_p)):
#         # print(t_p)
#         # 重新生成个体
#         num_zeros = random.randint(100, 110) # 确保至少有 80 个 0, 最多 113 个 0
#         num_non_zeros = 113 - num_zeros
#         non_zero_values = random.sample(range(1, 372), num_non_zeros)
#         zeros = random.choices([0], k=num_zeros)
#         t_p = non_zero_values + zeros

```

```

#         random.shuffle(t_p)

#         attempts += 1
#         if attempts >= max_attempts:
#             print("生成个体失败，达到最大重试次数")
#             break
#         pop.append(t_p)

# %%
# df_pop_new_init = pd.DataFrame(pop)
# df_pop_new_init.to_excel("pop_333.xlsx")

# %%
def crossover_and_mutation_rateFun(x,fitnesss,rate_max,rate_min):
    c0 = 9.903438
    fitvalue_min = min(fitnesss)
    fitvalue_avg = sum(fitnesss)/len(fitnesss)
    return (rate_max - rate_min) / (1 + np.exp(-c0 + 2 * c0 * (x - fitvalue_min) / (fitvalue_avg - fitvalue_min))) + rate_min

# %%
#导入一个已经初始化的数据
df_org = pd.read_excel("pop_333.xlsx")
df_org_list = df_org.values.tolist()
pop_111 = copy.deepcopy(df_org_list)

# %%
max_fit = []
for generation in range(N_GENERATIONS): # 迭代 N 代
    # 计算当前种群的适应度，传递当前代数和最大代数
    fitness = [get_fitness(pop_111[i]) for i in range(len(pop_111))]
    # 选择操作：使用锦标赛选择
    selected_pop, select_index = select(pop_111, fitness, tournament_size=4) # 从种群中选择适应度最好的个体
    # 交叉和变异操作
    CROSSOVER_RATE_change =
crossover_and_mutation_rateFun(fitness[select_index],fitness,0.8,0.1)
    MUTATION_RATE_change = crossover_and_mutation_rateFun(fitness[select_index],fitness,0.5,0.1)
    pop_111 = crossover_and_mutation(pop_111, select_index, CROSSOVER_RATE_change, MUTATION_RATE_change, fitness) # 交叉和变异生成新种群
    # 计算并打印当前代的适应度（可以查看最好的个体的适应度）
    fitness_true = [get_true_fitness(pop_111[i]) for i in range(len(pop_111))]
    best_fitness = max(fitness_true)
    max_fit.append(best_fitness)

```

```

print(f'Generation {generation + 1}: Best Fitness = {best_fitness}')

# %%
# 绘制折线图
plt.figure(figsize=(10, 6)) # 设置图形大小，宽 10，高 6
plt.plot(range(1, N_GENERATIONS + 1), max_fit, color='b')
plt.title('Best Fitness Over Generations')
plt.xlabel('Generation')
plt.ylabel('Best Fitness')
plt.grid(False) # 禁用背景的网格线
plt.show()

# %%
for g in pop_111:
    t = 0
    for h in g:
        if h != 0:
            t += 1
    print(t)

# %%
temp_output_df = pd.DataFrame(pop_111)
temp_output_df.to_excel("【test】居民满意度结果-种群数据-3 居民结果.xlsx")

# %%
move_result = [['原地块 ID','原院落 ID','原地块面积','地块采光','是否确定搬迁','搬到的地块','搬到的院落','搬迁后地块面积','迁后地块采光','面积变化','面积-满意度','采光变化','采光补偿-满意度','是否修缮','修缮金额','修缮-满意度']]

fitness_true = [get_true_fitness(pop_111[i]) for i in range(len(pop_111))]
pop_best_one = [pop_111[fitness_true.index(max(fitness_true))]]
for one_body in pop_best_one:#遍历整个种群中所有的个体
    # print(one_body)
    ttltl_data = translateDNA(one_body)
    for ned_mv_no in range(len(one_body)):
        rs_temp = []
        if one_body[ned_mv_no] != 0:
            target_mv_no = one_body[ned_mv_no]-1
        else:
            target_mv_no = 9999#表示不搬
        #原地块 ID
        rs_temp.append(dkID_i[ned_mv_no])
        #原院落 ID

```

```

rs_temp.append(yard_i[ned_mv_no])
#原地块面积
rs_temp.append(areas_i[ned_mv_no])
#地块采光
rs_temp.append(light_i[ned_mv_no])
#是否确定搬迁
if target_mv_no!=9999:
    rs_temp.append(1)
else:
    rs_temp.append(0)
#搬到的地块
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(dkID_j[target_mv_no])
#搬到的院落
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(yard_j[target_mv_no])
#搬迁后地块面积
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(areas_j[target_mv_no])
#迁后地块采光
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(light_j[target_mv_no])
#面积变化
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(areas_j[target_mv_no]-areas_i[ned_mv_no])
#面积-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(0.6*(areas_j[target_mv_no]-areas_i[ned_mv_no])/53)
#采光变化
if target_mv_no==9999:
    rs_temp.append(9999)
else:

```

```

        rs_temp.append(light_j[target_mv_no]-light_i[ned_mv_no])
#采光补偿-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(0.3*(light_j[target_mv_no]-light_i[ned_mv_no])/2)
#是否修缮
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    if
        areas_i[ned_mv_no]==areas_j[target_mv_no]
light_i[ned_mv_no]==light_j[target_mv_no]:
        rs_temp.append(1)
    else:
        rs_temp.append(0)
#修缮金额
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    if
        areas_i[ned_mv_no]==areas_j[target_mv_no]
light_i[ned_mv_no]==light_j[target_mv_no]:
        rs_temp.append(rcost*areas_j[target_mv_no])
    else:
        rs_temp.append(0)
#修缮-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    if
        areas_i[ned_mv_no]==areas_j[target_mv_no]
light_i[ned_mv_no]==light_j[target_mv_no]:
        rs_temp.append(0.1*rcost*areas_j[target_mv_no]/20)
    else:
        rs_temp.append(0)
move_result.append(rs_temp)

# %%
move_result_df = pd.DataFrame(move_result)
move_result_df.to_excel("3 因素全种群_move_result.xlsx")

```

问题 1：八因素作为目标函数的遗传算法

```
# %%  
import numpy as np  
import matplotlib.pyplot as plt  
import pandas as pd  
import random  
import copy  
  
# %%  
# 参数设置  
DNA_SIZE = 113 # DNA 大小 1-9 表示搬迁前 i 10-18 表示搬入 j 19 表示 xij 的值 20 表示 rj  
POP_SIZE = 50 # 种群大小  
CROSSOVER_RATE = 0.8 # 初始化交叉率  
MUTATION_RATE = 0.2 # 初始化变异率  
N_GENERATIONS = 40000 # 代数  
rcost = 0.2 #单位面积翻新成本，单位为万  
Time = 120 #搬迁时间 天  
w1 = 0.333 #面积补偿权重  
w2 = 0.197 #采光补偿权重  
w3 = 0.136 #修缮补偿权重  
w4 = 0.127 #邻里关系权重  
w5 = 0.063 #是否临近街道权重  
w6 = 0.083 #住户密度权重  
w7 = 0.028 #院落密集程度权重  
w8 = 0.032 #是否毗邻搬迁权重  
num_residents=113  
num_empty_houses=371  
M = 99999 # 极大常数  
  
# %%  
# 读取地块信息数据  
file_path = '附件一：老城街区地块信息.xlsx'  
data = pd.read_excel(file_path)  
# 提取相关数据  
areas = data['地块面积'].values # 地块面积  
orientations = data['地块方位'].apply(lambda x: 1 if x in ['南', '北'] else 0).values # 0 为东西，1 为南北  
has_residents = data['是否有住户'].values # 是否有住户  
light = data['地块方位'].apply(lambda x: 1 if x == '西' else (2 if x == '东' else 3)).values  
yards = data['院落 ID'].values  
dkIDs = data['地块 ID'].values  
  
areas_i = [] #需要搬迁的用户的房屋面积  
areas_j = [] #空房 房屋 面积
```

```

orientations_i = [] #需要搬迁 房屋 朝向
orientations_j = [] #空房 房屋 朝向
light_i = [] #需要搬迁 房屋 采光
light_j = [] #空房 房屋 采光
yard_i = [] #需要搬迁所属院落 ID
yard_j = [] #空房 院落 ID
dkID_i = [] #需要搬迁 房屋 地块 id
dkID_j = [] #空房 地块 id
for index in range(len(has_residents)):
    if has_residents[index]==1:
        areas_i.append(areas[index])
        orientations_i.append(orientations[index])
        light_i.append(light[index])
        yard_i.append(yards[index])
        dkID_i.append(dkIDs[index])
    elif has_residents[index] == 0:
        areas_j.append(areas[index])
        orientations_j.append(orientations[index])
        light_j.append(light[index])
        yard_j.append(yards[index])
        dkID_j.append(dkIDs[index])
    else:
        print("错误! ! ! ! ")

# %%
#创建每一个住户搬迁的可行解
res_move_all_possibility = [[0] for _ in range(num_residents)]
for rs in range(len(areas_i)):
    for avil_targ in range(len(areas_j)):
        if areas_j[avil_targ] > areas_i[rs]:
            if 1.3*areas_i[rs] > areas_j[avil_targ]:
                if light_j[avil_targ] >= light_i[rs]:
                    res_move_all_possibility[rs].append(avil_targ+1)

# %%
#院落数据
yard_data_df = pd.read_excel('院落信息.xlsx')#有 5 列数据: 包含地块 ID、地块个数、原始居住状态（住了几户人）、是否临街、顶点的度（相邻院落数量）
yard_data = yard_data_df.values.tolist()

# %%
#读取毗邻矩阵
close_yard_data_df = pd.read_excel('毗邻矩阵.xlsx')
close_yard_data = close_yard_data_df.values.tolist()

```



```

# %%
def Constraints_more(lst):
    # print(lst)
    # 只约束不重复的条件
    seen = set()
    for item in lst:
        if item != 0: # 忽略 0, 因为 0 表示不搬迁
            if item in seen: # 如果元素已经在集合中, 说明重复
                return False
            seen.add(item) # 将非零元素加入集合
    return True # 如果遍历完没有重复, 返回 True

def Constraints_s(lst):
    # 约束面积
    for n in range(num_residents):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 检查面积是否满足条件: areas_i[n] <= areas_j[lst[n]-1] 且 1.3 * areas_j[lst[n]-1] <=
areas_i[n]
            if areas_i[n] > areas_j[lst[n]-1] or areas_j[lst[n]-1] > 1.3 * areas_i[n]:
                # print(areas_i[n], 1.3 * areas_i[n], areas_j[lst[n]-1])
                return False
    return True # 如果遍历完没有问题, 返回 True

def Constraints_light(lst):
    # 约束采光
    for n in range(len(lst)):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 检查采光是否满足条件: light_j[lst[n]-1] >= light_i[n]
            if light_j[lst[n]-1] < light_i[n]:
                # print("新采光"+str(light_j[lst[n]-1]))
                # print("原采光"+str(light_i[n]))
                return False
    return True # 如果遍历完没有问题, 返回 True

def Constraints_rebuilt_money(lst):
    # 约束采光
    for n in range(len(lst)):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 判断重修花的钱是否超支, 首先要判断是否需要重修, 如果需要的话, 才判断花的钱
超支
            if areas_j[lst[n]-1] == areas_i[n] or light_j[lst[n]-1] == light_i[n]:
                rebuilt_money = rcost * areas_j[lst[n]-1]
                if rebuilt_money > 20:

```

```

        return False

    return True
def Constraints_sssss(lst):
    #约束面积负数
    for n in range(num_residents):
        if lst[n] != 0: # 如果该居民搬迁到地块
            if areas_i[n]>areas_j[lst[n]-1]:
                return False
    return True

# %%
def translateDNA(pop):
    #解码
    relocation_plan = {}
    for num in range(num_residents):
        target_house = pop[num] # 从当前个体（kkk）中获取每个居民的目标房子
        if target_house != 0: # 如果该居民搬迁到某个房子
            # 判断是否需要修缮
            rjjj = 1
            if areas_j[target_house-1] > areas_i[num] and orientations_j[target_house-1] >
orientations_i[num]:
                rjjj = 0
            relocation_plan[num] = {
                "i": num + 1, # 居民编号（从 1 开始）
                "j": target_house,
                "si": areas_i[num], # 假设房子编号从 1 开始，取相应的面积数据
                "sj": areas_j[target_house-1], # 目标面积数据
                "yi":yard_i[num],
                "yj":yard_j[target_house-1],
                "oi": orientations_i[num], # 同理取相应的朝向数据
                "oj": orientations_j[target_house-1], # 获取目标朝向
                "xij": 1,
                "rj": rjjj # 根据面积、朝向等信息决定是否需要修缮
            }
        else:
            # 等于 0 的时候，就是不搬的情况
            relocation_plan[num] = {
                "i": num + 1, # 居民编号（从 1 开始）
                "j": 999,
                "si": areas_i[num], # 假设房子编号从 1 开始，取相应的面积数据
                "sj": 999, # 目标面积数据
                "yi":yard_i[num],
                "yj":999,

```

```

        "oi": orientations_i[num], # 同理取相应的朝向数据
        "oj": 999, # 获取目标朝向
        "xij": 0,
        "tj": 0 # 这里可以根据面积、朝向等信息决定是否需要修缮
    }
    return relocation_plan

# %%
# def adaptive_penalty(x, iteration, max_iterations):
#     # 计算约束违反的程度
#     constraint_violation = 0

#     # 计算违反的约束条件，累加每个违反的约束值
#     if not Constraints_more(x): # 如果违反了不重复约束
#         constraint_violation += 1
#     if not Constraints_s(x): # 如果违反了面积约束
#         constraint_violation += 1
#     if not Constraints_light(x): # 如果违反了采光约束
#         constraint_violation += 1

#     # 自适应惩罚因子，随着迭代次数增加，惩罚逐渐增大
#     penalty_factor = 1 + (iteration / max_iterations) # 惩罚因子逐渐增大
#     return penalty_factor * constraint_violation # 返回惩罚值

# %%
# 写函数计算邻里关系，邻里关系保持不变的数量
def same_neibors(tl_data):
    same_neibor = 0
    before_move = []
    after_move = []
    for num in range(num_residents):
        if tl_data[num]['xij'] == 1:
            before_move.append(tl_data[num]['yi'])
            after_move.append(tl_data[num]['yj'])
    only_one = list(set(before_move)) # 计算有多少个不相同的院落
    for y in only_one:
        temp_index = [] # 先统计搬迁前是同一个院落的住户的 index 也就是 k
        for k in range(len(before_move)):
            if before_move[k] == y:
                temp_index.append(k)
        same_neibor += len([after_move[oo] for oo in temp_index]) - len(list(set([after_move[oo] for oo in temp_index]))) # 直接获取搬迁后用户所在院落，然后再用 set 去重，再用 list-set 获取住在同一个院落
    的数量

```

```

        return same_neibor
#判断是否临街
def close_to_steet_list(tl_data):
    #判断前后相见
    t = []
    for num in range(num_residents):
        if tl_data[num]['xij'] == 1:
            t1 = yard_data[tl_data[num]['yj']-1][3]-yard_data[tl_data[num]['yi']-1][3]
            t.append(t1)
        else:
            t.append(0)
    return t
#判断住户密度
def res_density_list(tl_data):
    densechange = []
    changed_dese = []
    for kkk in range(len(yard_data)):
        changed_dese.append([kkk,yard_data[kkk][1],yard_data[kkk][2]])
    # print(changed_dese)

    for num in range(num_residents):
        if tl_data[num]['xij'] == 1:
            for mmm in range(len(changed_dese)):
                if changed_dese[mmm][0] == tl_data[num]['yi']:
                    changed_dese[mmm][2] -=1
                    changed_dese[tl_data[num]['yj']-1][2] +=1
    for num2 in range(num_residents):
        if tl_data[num2]['xij'] == 1:
            densechange.append(yard_data[tl_data[num2]['yi']-1][2]/yard_data[tl_data[num2]['yi']-1][1]-changed_dese
[tl_data[num2]['yj']-1][2]/changed_dese[tl_data[num2]['yj']-1][1])
        else:
            densechange.append(0)
    return densechange
#院落密集程度
def yard_density_list(tl_data):
    ttt = []
    for num3 in range(num_residents):
        if tl_data[num3]['xij'] == 1:
            ttt.append(yard_data[tl_data[num3]['yi']-1][4]-yard_data[tl_data[num3]['yj']-1][4])
        else:
            ttt.append(0)
    return ttt
#是否毗邻搬迁

```

```

def is_closeyard_move_list(tl_data):
    t4t = []
    for num4 in range(num_residents):
        if tl_data[num4]['xij'] == 1:
            t4t.append(close_yard_data[tl_data[num4]['yi']-1][tl_data[num4]['yj']-1])
        else:
            t4t.append(0)
    return t4t

# %%
def F(tl_data):
    # 目标函数，包括新增住房面积 0.6、采光补偿 0.1、修缮补偿 0.3
    z = 0
    same_neibors_data = same_neibors(tl_data)
    close_to_steet_data = close_to_steet_list(tl_data)
    res_density_data = res_density_list(tl_data)
    yard_density_data = yard_density_list(tl_data)
    is_closeyard_move_data = is_closeyard_move_list(tl_data)
    for num in range(num_residents):
        if tl_data[num]['xij'] == 1: # 如果搬迁了
            z += w1*(tl_data[num]['sj']-tl_data[num]['si'])/53 +
w2*(light_j[tl_data[num]['j']-1]-light_i[num])/2 + w3*rcost*tl_data[num]['sj']*tl_data[num]['rj']/20+
w5*close_to_steet_data[num] + w6*(res_density_data[num]+0.94)/3.94+
w7*(yard_density_data[num]+5)/10+w8*is_closeyard_move_data[num]
            z+=w4*same_neibors_data
    return z

# %%
def get_fitness(pop_1, iteration, max_iterations):
    # 计算适应度
    tl_data = translateDNA(pop_1) # 获取 x, y 值
    # 计算目标函数，包括适应度和约束惩罚
    pred = F(tl_data) # 传入当前代数和最大代数，计算适应度（包括惩罚）
    # pred_penalty = pred-adaptive_penalty(pop_1, iteration, max_iterations) # 计算惩罚
    return pred

# %%
def get_true_fitness(pop_12, iteration, max_iterations):
    # 计算适应度
    tl_data = translateDNA(pop_12) # 获取 x, y 值
    # 计算目标函数，包括适应度和约束惩罚
    pred = F(tl_data) # 传入当前代数和最大代数，计算适应度（包括惩罚）
    return pred

```

```

# %%
def select(pop, fitness, tournament_size=4):
    # 随机选择 `tournament_size` 个个体进行锦标赛
    selected_indices = random.sample(range(len(pop)), tournament_size)
    #selected_indices = [1,2,3,4]

    # 选择适应度最好的个体
    select_fitness = [fitness[p] for p in selected_indices]
    select_index = fitness.index(min(select_fitness))
    # best_index = selected_indices[0]
    # for index in selected_indices[1:]:
    #     if fitness[index] > fitness[best_index]:
    #         best_index = index

    # selected=pop[best_index]

    return pop[select_index],select_index

# %%
def crossover(pop_4, parent1, parent2, crossover_rate):
    pop_5 = pop_4.copy() # 创建一个种群的副本

    if random.random() > crossover_rate:
        # 如果不进行交叉，直接返回父母
        return pop_5

    # 随机选择两个交叉点
    crossover_points = sorted(random.sample(range(1, DNA_SIZE), 2)) # 随机选择两个交叉点
    crossover_point1, crossover_point2 = crossover_points

    # 获取父母的交叉点基因值
    org_parent1_point1_value = pop_4[parent1][crossover_point1]
    org_parent1_point2_value = pop_4[parent1][crossover_point2]
    org_parent2_point1_value = pop_4[parent2][crossover_point1]
    org_parent2_point2_value = pop_4[parent2][crossover_point2]

    # 交换父母基因
    pop_5[parent1][crossover_point1] = org_parent2_point1_value
    pop_5[parent1][crossover_point2] = org_parent2_point2_value
    pop_5[parent2][crossover_point1] = org_parent1_point1_value
    pop_5[parent2][crossover_point2] = org_parent1_point2_value

```

```

# 交叉后，检查是否满足约束条件
attempts = 0
max_attempts = 40000 # 最大尝试次数，避免死循环

while not (Constraints_more(pop_5[parent1]) and Constraints_s(pop_5[parent1]) and
Constraints_light(pop_5[parent1]) and Constraints_rebuilt_money(pop_5[parent1]) and
Constraints_more(pop_5[parent2]) and Constraints_s(pop_5[parent2]) and
Constraints_light(pop_5[parent2]) and Constraints_rebuilt_money(pop_5[parent2])):
    # 如果不满足约束条件，重新选择交叉点并进行交换
    pop_5 = pop_4.copy() # 创建一个种群的副本
    crossover_points = sorted(random.sample(range(1, DNA_SIZE), 2)) # 重新随机选择两个交叉点

    crossover_point1, crossover_point2 = crossover_points

    # 获取新的交叉点基因值
    org_parent1_point1_value = pop_4[parent1][crossover_point1]
    org_parent1_point2_value = pop_4[parent1][crossover_point2]
    org_parent2_point1_value = pop_4[parent2][crossover_point1]
    org_parent2_point2_value = pop_4[parent2][crossover_point2]

    # 交换父母基因
    pop_5[parent1][crossover_point1] = org_parent2_point1_value
    pop_5[parent1][crossover_point2] = org_parent2_point2_value
    pop_5[parent2][crossover_point1] = org_parent1_point1_value
    pop_5[parent2][crossover_point2] = org_parent1_point2_value

    attempts += 1
    if attempts >= max_attempts:
        print("交换无法生成符合约束条件的个体，已达到最大尝试次数。")
        break # 如果超过最大尝试次数，则退出循环
if attempts < max_attempts:
    return pop_5
else:
    return pop_4

# %%
def crossover(pop_4, parent1, parent2, crossover_rate):
    pop_5 = copy.deepcopy(pop_4) # 创建一个种群的副本

    if random.random() > crossover_rate:
        # 如果不进行交叉，直接返回父母
        return pop_5

```

```

# 随机选择两个交叉点
crossover_points = sorted(random.sample(range(1, DNA_SIZE), 2)) # 随机选择两个交叉点
crossover_point1, crossover_point2 = crossover_points

# 获取父母的交叉点基因值
org_parent1_point1_value = pop_4[parent1][crossover_point1]
org_parent1_point2_value = pop_4[parent1][crossover_point2]
org_parent2_point1_value = pop_4[parent2][crossover_point1]
org_parent2_point2_value = pop_4[parent2][crossover_point2]

# 交换父母基因
pop_5[parent1][crossover_point1] = org_parent2_point1_value
pop_5[parent1][crossover_point2] = org_parent2_point2_value
pop_5[parent2][crossover_point1] = org_parent1_point1_value
pop_5[parent2][crossover_point2] = org_parent1_point2_value
if Constraints_more(pop_5[parent1]) and Constraints_s(pop_5[parent1]) and
Constraints_light(pop_5[parent1]) and Constraints_rebult_money(pop_5[parent1]) and
Constraints_more(pop_5[parent2]) and Constraints_s(pop_5[parent2]) and
Constraints_light(pop_5[parent2]) and Constraints_rebult_money(pop_5[parent2]):
    return pop_5
else:
    return pop_4

# %%
def mutation(pop_7, parent1_num, mutation_rate):
    pop_8 = copy.deepcopy(pop_7) # 创建副本，避免直接修改原种群

    # 如果随机数小于变异率，则不进行变异，直接返回
    if random.random() > mutation_rate:
        return pop_7

    # 进行变异：选择一个随机的基因位置
    mutation_point = random.randint(0, len(pop_8[0]) - 1)

    # 变异为一个随机的地块编号
    # pop_8[parent1_num][mutation_point] = random.randint(0, 371)
    pop_8[parent1_num][mutation_point] = random.sample(res_move_all_possibility[mutation_point], 1)[0]

    # 如果不满足约束条件，重新变异
    if Constraints_more(pop_8[parent1_num]) and Constraints_s(pop_8[parent1_num]) and
Constraints_light(pop_8[parent1_num]) and Constraints_rebult_money(pop_8[parent1_num]):
        return pop_8
    else:

```



```

        return pop_7

# %%
def crossover_and_mutation_rateFun(x, fitnesss, rate_max, rate_min):
    c0 = 9.903438
    fitvalue_min = min(fitnesss)
    fitvalue_avg = sum(fitnesss)/len(fitnesss)
    return (rate_max - rate_min) / (1 + np.exp(-c0 + 2 * c0 * (x - fitvalue_min) / (fitvalue_avg - fitvalue_min))) + rate_min

# %%
# pop = []
# for p in range(POP_SIZE):
#     # 确保至少有 80 个 0，最多 113 个 0
#     num_zeros = random.randint(100, 110) # 随机选择生成多少个 0，范围在 80 到 113 之间
#     num_non_zeros = 113 - num_zeros # 计算剩余的非零数字数量
#     # 生成不重复的其他数字（从 1 到 371）共生成 num_non_zeros 个
#     non_zero_values = random.sample(range(1, 372), num_non_zeros)
#     # 使用 random.choices 插入重复的 0
#     zeros = random.choices([0], k=num_zeros) # 生成 num_zeros 个 0
#     # 合并不重复的数字和 0
#     t_p = non_zero_values + zeros
#     # 打乱列表，确保 0 不固定在前或后
#     random.shuffle(t_p)

#     # 使用 not 来确保在约束条件不满足时重新生成个体
#     attempts = 0
#     max_attempts = 1000000 # 设置最大尝试次数以防止死循环
#     while not (Constraints_more(t_p) and Constraints_s(t_p) and Constraints_light(t_p) and
Constraints_rebult_money(t_p)):
#         # print(t_p)
#         # 重新生成个体
#         num_zeros = random.randint(100, 110) # 确保至少有 80 个 0，最多 113 个 0
#         num_non_zeros = 113 - num_zeros
#         non_zero_values = random.sample(range(1, 372), num_non_zeros)
#         zeros = random.choices([0], k=num_zeros)
#         t_p = non_zero_values + zeros
#         random.shuffle(t_p)

#         attempts += 1
#         if attempts >= max_attempts:
#             print("生成个体失败，达到最大重试次数")
#             break
#     pop.append(t_p)

```

```

# %%
# df_pop_new_init = pd.DataFrame(pop)
# df_pop_new_init.to_excel("pop_222.xlsx")

# %%
# 导入一个已经初始化的数据
df_org = pd.read_excel("pop_333.xlsx")
df_org_list = df_org.values.tolist()

# %%
pop_111 = df_org_list.copy()

# %%
max_fit = []
for generation in range(N_GENERATIONS): # 迭代 N 代
    # 计算当前种群的适应度，传递当前代数和最大代数
    fitness = [get_fitness(pop_111[i], generation, N_GENERATIONS) for i in range(len(pop_111))]
    # 选择操作：使用锦标赛选择
    selected_pop, select_index = select(pop_111, fitness, tournament_size=4) # 从种群中选择适应度
    最好的个体
    # 交叉和变异操作
    CROSSOVER_RATE_change =
crossover_and_mutation_rateFun(fitness[selected_index], fitness, 0.7, 0.1)
    MUTATION_RATE_change = crossover_and_mutation_rateFun(fitness[selected_index], fitness, 0.3, 0.1)
    pop_111 = crossover_and_mutation(pop_111, select_index, CROSSOVER_RATE_change,
MUTATION_RATE_change, fitness) # 交叉和变异生成新种群
    # 计算并打印当前代的适应度（可以查看最好的个体的适应度）
    fitness_true = [get_true_fitness(pop_111[i], generation, N_GENERATIONS) for i in
range(len(pop_111))]
    best_fitness = max(fitness_true)
    max_fit.append(best_fitness)
    print(f"Generation {generation + 1}: Best Fitness = {best_fitness}")

# %%
# 绘制折线图
plt.figure(figsize=(10, 6)) # 设置图形大小，宽 10，高 6
plt.plot(range(1, N_GENERATIONS + 1), max_fit, color='b')
plt.title('Best Fitness Over Generations')
plt.xlabel('Generation')
plt.ylabel('Best Fitness')
plt.grid(False) # 禁用背景的网格线
plt.show()

```

```

# %%
for g in pop_111:
    t = 0
    for h in g:
        if h != 0:
            t += 1
    print(t)

# %%
move_result = [['原地块 ID','原院落 ID','原地块面积','原地块采光','是否确定搬迁','搬到的地块','搬到的院落','搬迁后地块面积','迁后地块采光','面积变化','面积-满意度','采光变化','采光补偿-满意度','是否修缮','修缮金额','修缮-满意度','临街变化','临街变化-满意度','住户密度变化','住户密度变化-满意度','院落密度变化','院落密度变化-满意度','是否毗邻搬迁','毗邻搬迁-满意度','邻里关系']]

fitness_true = [get_true_fitness(pop_111[i], generation, N_GENERATIONS) for i in range(len(pop_111))]
pop_best_one = [pop_111[fitness_true.index(max(fitness_true))]]
for one_body in pop_best_one: #遍历整个种群中所有的个体

    ttttl_data = translateDNA(one_body)
    close_to_steet_list_ = close_to_steet_list(ttttl_data)
    res_density_list_ = res_density_list(ttttl_data)
    yard_density_list_ = yard_density_list(ttttl_data)
    is_closeyard_move_list_ = is_closeyard_move_list(ttttl_data)
    same_neibors_d = same_neibors(ttttl_data)
    for ned_mv_no in range(len(one_body)):
        rs_temp = []
        if one_body[ned_mv_no] != 0:
            target_mv_no = one_body[ned_mv_no]-1
        else:
            target_mv_no = 9999 #表示不搬
        #原地块 ID
        rs_temp.append(dkID_i[ned_mv_no])
        #原院落 ID
        rs_temp.append(yard_i[ned_mv_no])
        #原地块面积
        rs_temp.append(areas_i[ned_mv_no])
        #地块采光
        rs_temp.append(light_i[ned_mv_no])
        #是否确定搬迁
        if target_mv_no != 9999:
            rs_temp.append(1)
        else:
            rs_temp.append(0)

```

```

#搬到的地块
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(dkID_j[target_mv_no])
#搬到的院落
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(yard_j[target_mv_no])
#搬迁后地块面积
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(areas_j[target_mv_no])
#迁后地块采光
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(light_j[target_mv_no])
#面积变化
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(areas_j[target_mv_no]-areas_i[ned_mv_no])
#面积-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(w1*(areas_j[target_mv_no]-areas_i[ned_mv_no])/53)
#采光变化
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(light_j[target_mv_no]-light_i[ned_mv_no])
#采光补偿-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(w2*(light_j[target_mv_no]-light_i[ned_mv_no])/2)
#是否修缮
if target_mv_no==9999:
    rs_temp.append(9999)
else:

```

```

        if areas_i[ned_mv_no]==areas_j[target_mv_no] or
light_i[ned_mv_no]==light_j[target_mv_no]:
            rs_temp.append(1)
        else:
            rs_temp.append(0)
        #修缮金额
        if target_mv_no==9999:
            rs_temp.append(9999)
        else:
            if areas_i[ned_mv_no]==areas_j[target_mv_no] or
light_i[ned_mv_no]==light_j[target_mv_no]:
                rs_temp.append(rcost*areas_j[target_mv_no])
            else:
                rs_temp.append(0)
        #修缮-满意度
        if target_mv_no==9999:
            rs_temp.append(9999)
        else:
            if areas_i[ned_mv_no]==areas_j[target_mv_no] or
light_i[ned_mv_no]==light_j[target_mv_no]:
                rs_temp.append(w3*rcost*areas_j[target_mv_no]/20)
            else:
                rs_temp.append(0)
        #临街变化
        if target_mv_no==9999:
            rs_temp.append(9999)
        else:
            rs_temp.append(close_to_steet_list_[ned_mv_no])
        #临街变化-满意度
        if target_mv_no==9999:
            rs_temp.append(9999)
        else:
            rs_temp.append(w5*close_to_steet_list_[ned_mv_no])
        #住户密度变化
        if target_mv_no==9999:
            rs_temp.append(9999)
        else:
            rs_temp.append(res_density_list_[ned_mv_no])
        #住户密度变化-满意度
        if target_mv_no==9999:
            rs_temp.append(9999)
        else:
            rs_temp.append(w6*res_density_list_[ned_mv_no])
        #院落密度变化

```

```

if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(yard_density_list_[ned_mv_no])
#院落密度变化-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(w7*yard_density_list_[ned_mv_no])
#是否毗邻搬迁
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(is_closeyard_move_list_[ned_mv_no])
#毗邻搬迁-满意度
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(w8*is_closeyard_move_list_[ned_mv_no])
#邻里关系
if target_mv_no==9999:
    rs_temp.append(9999)
else:
    rs_temp.append(same_neibors_d)

move_result.append(rs_temp)

# %%
move_result_df = pd.DataFrame(move_result)
move_result_df.to_excel("8 因素_move_result-40000.xlsx")

# %%

```

问题二：多目标函数的 NSGA-II 算法

```
# %%  
import numpy as np  
import pandas as pd  
import random  
import copy  
  
# %%  
# 参数设置  
DNA_SIZE = 113 # DNA 大小 1-9 表示搬迁前 i 10-18 表示搬入 j 19 表示 xij 的值 20 表示 rj  
POP_SIZE = 50 # 种群大小  
CROSSOVER_RATE = 0.8 # 初始化交叉率  
MUTATION_RATE = 0.2 # 初始化变异率  
N_GENERATIONS = 40000 # 代数  
rcost = 0.2 # 单位面积翻新成本，单位为万  
Time = 120 # 搬迁时间 天  
w1 = 0.6 # 面积补偿权重  
w2 = 0.3 # 采光补偿权重  
w3 = 0.1 # 修缮补偿权重  
num_residents = 113  
num_empty_houses = 371  
  
# %%  
# 读取地块信息数据  
file_path = '附件一：老城街区地块信息.xlsx'  
data = pd.read_excel(file_path)  
# 提取相关数据  
areas = data['地块面积'].values # 地块面积  
orientations = data['地块方位'].apply(lambda x: 1 if x in ['南', '北'] else 0).values # 0 为东西，1 为南北  
has_residents = data['是否有住户'].values # 是否有住户  
light = data['地块方位'].apply(lambda x: 1 if x == '西' else (2 if x == '东' else 3)).values  
yards = data['院落 ID'].values  
dkIDs = data['地块 ID'].values  
yard_s = data['地块面积'].values  
  
areas_i = [] # 需要搬迁的用户的房屋面积  
areas_j = [] # 空房 房屋 面积  
orientations_i = [] # 需要搬迁 房屋 朝向  
orientations_j = [] # 空房 房屋 朝向  
light_i = [] # 需要搬迁 房屋 采光  
light_j = [] # 空房 房屋 采光  
yard_i = [] # 需要搬迁所属院落 ID  
yard_j = [] # 空房 院落 ID
```

```

dkID_i = [] #需要搬迁 房屋 地块 id
dkID_j = [] #空房 地块 id
for index in range(len(has_residents)):
    if has_residents[index]==1:
        areas_i.append(areas[index])
        orientations_i.append(orientations[index])
        light_i.append(light[index])
        yard_i.append(yards[index])
        dkID_i.append(dkIDs[index])
    elif has_residents[index] == 0:
        areas_j.append(areas[index])
        orientations_j.append(orientations[index])
        light_j.append(light[index])
        yard_j.append(yards[index])
        dkID_j.append(dkIDs[index])
    else:
        print("错误！！！！！")

# %%
#院落数据
yard_data_df = pd.read_excel('院落信息.xlsx')#有 5 列数据：包含地块 ID、地块个数、原始居住状态（住
了几户人）、是否临街、顶点的度（相邻院落数量）
yard_data = yard_data_df.values.tolist()

# %%
#读取毗邻矩阵
close_yard_data_df = pd.read_excel('毗邻矩阵.xlsx')
close_yard_data = close_yard_data_df.values.tolist()

# %%
#创建每一个住户搬迁的可行解
res_move_all_possibility = [[0] for _ in range(num_residents)]
for rs in range(len(areas_i)):
    for avil_targ in range(len(areas_j)):
        if areas_j[avil_targ] > areas_i[rs]:
            if 1.3*areas_i[rs] > areas_j[avil_targ]:
                if light_j[avil_targ] >= light_i[rs]:
                    res_move_all_possibility[rs].append(avil_targ+1)

# res_move_all_possibility_df = pd.DataFrame(res_move_all_possibility)
# res_move_all_possibility_df.to_excel("res_move_all_possibility.xlsx")

# %%
def Constraints_more(lst):

```



```

# print(lst)
# 只约束不重复的条件
seen = []
for item in lst:
    if item != 0: # 忽略 0, 因为 0 表示不搬迁
        if item in seen: # 如果元素已经在集合中, 说明重复
            return False
        seen.append(item) # 将非零元素加入集合
return True # 如果遍历完没有重复, 返回 True

def Constraints_s(lst):

    # 约束面积
    for n in range(num_residents):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 检查面积是否满足条件: areas_i[n] <= areas_j[lst[n]-1] 且 1.3 * areas_j[lst[n]-1] <=
areas_i[n]
            if not (areas_i[n] <= areas_j[lst[n]-1] and areas_j[lst[n]-1] <= 1.3 * areas_i[n]):
                # print(str(n+1)+"->" + str(lst[n]))
                # print("原面积:" + str(areas_i[n]))
                # print("新面积:" + str(areas_j[lst[n]-1]))
                # print("原面积*130%:" + str(1.3 * areas_i[n]))
                return False
    return True # 如果遍历完没有问题, 返回 True

def Constraints_ssssss(lst):
    # 约束面积负数
    for n in range(num_residents):
        if lst[n] != 0: # 如果该居民搬迁到地块
            if areas_i[n] > areas_j[lst[n]-1]:
                return False
    return True

def Constraints_light(lst):
    # 约束采光
    for n in range(len(lst)):
        if lst[n] != 0: # 如果该居民搬迁到地块
            # 检查采光是否满足条件: light_j[lst[n]-1] >= light_i[n]
            if light_j[lst[n]-1] < light_i[n]:
                # print("新采光" + str(light_j[lst[n]-1]))
                # print("原采光" + str(light_i[n]))
                return False
    return True # 如果遍历完没有问题, 返回 True

```

```

def Constraints_rebuilt_money(lst):
    # 约束采光
    for n in range(len(lst)):
        if lst[n] != 0: # 如果该居民搬迁到地块
            #判断重修花的钱是否超支，首先要判断是否需要重修，如果需要的话，才判断花的钱
            超支
            if areas_j[lst[n]-1] == areas_i[n] or light_j[lst[n]-1] == light_i[n]:
                rebuilt_money = rcost*areas_j[lst[n]-1]
                if rebuilt_money>20:
                    return False
    return True

# %%
def translateDNA(pop):
    #解码
    relocation_plan = {}
    for num in range(num_residents):
        target_house = pop[num] # 从当前个体（kkk）中获取每个居民的目标房子
        if target_house != 0: # 如果该居民搬迁到某个房子
            # 判断是否需要修缮
            rjjj = 1
            if areas_j[target_house-1] > areas_i[num] and orientations_j[target_house-1] >
            orientations_i[num]:
                rjjj = 0
            relocation_plan[num] = {
                "i": num + 1, # 居民编号（从 1 开始）
                "j": target_house,
                "si": areas_i[num], # 假设房子编号从 1 开始，取相应的面积数据
                "sj": areas_j[target_house-1], # 目标面积数据
                "yi": yard_i[num],
                "yj": yard_j[target_house-1],
                "oi": orientations_i[num], # 同理取相应的朝向数据
                "oj": orientations_j[target_house-1], # 获取目标朝向
                'dki': dkID_i[num],
                'dkj': dkID_j[target_house-1],
                "xij": 1,
                "rj": rjjj # 根据面积、朝向等信息决定是否需要修缮
            }
        else:
            # 等于 0 的时候，就是不搬的情况
            relocation_plan[num] = {
                "i": num + 1, # 居民编号（从 1 开始）
                "j": 999,
                "si": areas_i[num], # 假设房子编号从 1 开始，取相应的面积数据

```

```

        "sj": 999, # 目标面积数据
        "yi": yard_i[num],
        "yj": 999,
        "oi": orientations_i[num], # 同理取相应的朝向数据
        "oj": 999, # 获取目标朝向
        'dk': dkID_i[num],
        'dkj': 999,
        "xij": 0,
        "rj": 0 # 这里可以根据面积、朝向等信息决定是否需要修缮
    }
    return relocation_plan

# %%
def F_res_satisfie(tl_data):
    # 目标函数，包括新增住房面积 0.6、采光补偿 0.1、修缮补偿 0.3
    z = 0
    for num in range(num_residents):
        if tl_data[num]["xij"] == 1: # 如果搬迁了
            z
            += 0.6*(tl_data[num]["sj"]-tl_data[num]["si"])/53+0.1*rcost*tl_data[num]["sj"]*tl_data[num]["rj"]/20 +
            0.3*(light_j[tl_data[num]["j"]]-1)-light_i[num])/2
    return z

def F_movenum(tl_data):
    movePeople = 0
    for num in range(num_residents):
        if tl_data[num]["xij"] == 1: # 如果搬迁了
            movePeople -= 1
    return movePeople

def F_fullYard(tl_data):
    fullYard_rent = 0
    #创建一个字典，从 0-107 作为编号，包含院落 ID、顶点的度和住户数量
    fullYard_dic = {}
    for yd in range(len(yard_data)):
        fullYard_dic[yd+1] = {
            'du': yard_data[yd][4],
            'resnum': 0,
            's': 0
        }
    for ttt in range(len(yards)):
        fullYard_dic[yards[ttt]]['s'] = yard_s[ttt]
    for num in range(num_residents):
        if tl_data[num]["xij"] == 1: # 如果搬迁了

```

```

        fullYard_dic[tl_data[num]['yj']]['resnum']+=1
    else:
        fullYard_dic[tl_data[num]['yi']]['resnum']+=1
for dt in fullYard_dic:
    if fullYard_dic[dt]['resnum']==0:
        if fullYard_dic[dt]['du']>0:
            fullYard_rent+=1.2*fullYard_dic[dt]['s']*30
        else:
            fullYard_rent+=1.0*fullYard_dic[dt]['s']*30
return fullYard_rent

# %%
def non_dominated_sort(pops):
    # Step 1: 计算每个个体的目标值
    F_res_satisfie_data = [] #每一个个体中居民满意度数据
    F_movenum_data = [] #每一个个体中搬迁人数数据
    F_fullYard_data = [] #每一个个体中满院落数据
    for one_pop in pops:
        tl_non_data = translateDNA(one_pop)
        F_res_satisfie_data.append(F_res_satisfie(tl_non_data))
        F_movenum_data.append(F_movenum(tl_non_data))
        F_fullYard_data.append(F_fullYard(tl_non_data))
    # Step 2: 初始化个体的支配集合和被支配次数
    n = len(pops)
    dominate_and_dominatedtimes = [[[0] for k in range(n)]]
    rank = [0 for kk in range(n)]
    QY_V = [[]] # 用于存储非支配前沿集合 用处?
    # Step 3: 比较每对个体的支配关系
    for i in range(n):
        for j in range(i + 1, n):
            p = dominate_and_dominatedtimes[i]
            q = dominate_and_dominatedtimes[j]
            # 计算目标值并比较支配关系
            p_values = [F_res_satisfie_data[i], F_movenum_data[i], F_fullYard_data[i]]
            q_values = [F_res_satisfie_data[j], F_movenum_data[j], F_fullYard_data[j]]
            # 判断 p 是否支配 q, 或者 q 是否支配 p
            if all(pv >= qv for pv, qv in zip(p_values, q_values)) and any(pv > qv for pv, qv in
zip(p_values, q_values)):
                p[0].append(j)#p 支配 q, 把 q 放在 p 的支配集合里
                q[1]+=1#q 被支配次数+=1
            elif all(qv >= pv for pv, qv in zip(p_values, q_values)) and any(qv > pv for pv, qv in
zip(p_values, q_values)):
                q[0].append(i)#q 支配 p, 把 p 放在 q 的支配集合中
                p[1]+=1

```

```

        if p[1] == 0:
            QY_V[0].append(i)
            rank[i] = 1 # 设置非支配等级为 1
# Step 4: 构造后续的非支配前沿
k=0
while True:
    Q = []#存储下一层前沿的个体索引
    for y in QY_V[k]:
        n = dominate_and_dominatedtimes[y]
        for yy in n[0]:
            dominate_and_dominatedtimes[yy][1]-=1 # 去掉 p 对 q 的支配
            if dominate_and_dominatedtimes[yy][1] == 0:
                Q.append(yy) # 添加到下一层前沿
                rank[yy]=k+2
    if not Q:
        break # 如果没有更多个体加入，退出循环
    else:
        QY_V.append(Q) # 将下一层前沿添加到 QY_V 中
        k+=1 # 进入下一层
# print(QY_V)
return QY_V,rank

# %%
def Cl_cd(pops,QY_V):
    cd = [0 for g in range(len(pops))]
    # Step 1: 计算每个个体的目标值
    F_res_satisfie_data = [] #每一个个体中居民满意度数据
    F_movenum_data = [] #每一个个体中搬迁人数数据
    F_fullYard_data = [] #每一个个体中满院落数据
    for one_pop in pops:
        tl_non_data = translateDNA(one_pop)
        F_res_satisfie_data.append(F_res_satisfie(tl_non_data))
        F_movenum_data.append(F_movenum(tl_non_data))
        F_fullYard_data.append(F_fullYard(tl_non_data))
    all_func = [F_res_satisfie_data,F_movenum_data,F_fullYard_data]
    nf = len(QY_V)#获取前沿的层数
    for i in range(nf):#循环处理 QY_V 中的数据
        n = len(QY_V[i])#获取第 i 层前沿（QY_V）中的个体数
        nobj = 3 #目标函数维度
        d = [[0]*nobj for uu in range(n)]#初始化每个个体在目标函数上的拥挤距离，这里是 n×3，n
        行 3 列
        for j in range(nobj):
            val = np.sort([all_func[j][t] for t in QY_V[i]])#对居民满意度 F_res_satisfie_data 的函数值

```

从小到大进行排序

ind = np.argsort([all_func[j][t] for t in QY_V[i]])#获取排序后的索引,比如第一层有 7 个,指的是在 7 个里面的所有, 0-6

d[ind[0][0]][j] = 999999 #对边界赋值无穷大 因为 ind 排序后的索引指的是 0-n, 所以不是 pop 中真正的所以, 这里需要调用 QY_V 来获取原 pop 中的索引

d[ind[-1][0]][j] = 999999 #对边界赋值无穷大

for r in range(1,n-1):

对中间个体: 计算前后个体目标值的差值 / 总范围, 实现归一化

d[ind[r]][j] = abs(val[r + 1] - val[r - 1])/abs(val[-1] - val[0])

for m in range(n):

#将每个个体在各目标上的归一化拥挤度相加, 作为总拥挤度

cd[QY_V[i][m]] = sum(d[m])

return cd

%%

#排序

def multi_sort(pops):

QY_V,Rank = non_dominated_sort(pops)

CD = Cl_cd(pops,QY_V,)

rank_new = []

copy_pop = pops.copy()

new_pop = []

#根据拥挤度 cd 进行排序

for cdd in np.argsort(CD)[::-1]:

new_pop.append(copy_pop[cdd])

rank_new.append(Rank[cdd])

new_new_pop = []

#根据非支配等级进行升序, rank 小的排前面

for rkk in np.argsort(rank_new):

new_new_pop.append(new_pop[rkk])

return new_new_pop

%%

def crossover(pop_4, parent1, parent2, crossover_rate):

pop_5 = copy.deepcopy(pop_4) # 创建一个种群的副本

if random.random() > crossover_rate:

如果不进行交叉, 直接返回父母

return pop_5

随机选择两个交叉点

crossover_points = sorted(random.sample(range(1, DNA_SIZE), 2)) # 随机选择两个交叉点

crossover_point1, crossover_point2 = crossover_points

```

# 获取父母的交叉点基因值
org_parent1_point1_value = pop_4[parent1][crossover_point1]
org_parent1_point2_value = pop_4[parent1][crossover_point2]
org_parent2_point1_value = pop_4[parent2][crossover_point1]
org_parent2_point2_value = pop_4[parent2][crossover_point2]

# 交换父母基因
pop_5[parent1][crossover_point1] = org_parent2_point1_value
pop_5[parent1][crossover_point2] = org_parent2_point2_value
pop_5[parent2][crossover_point1] = org_parent1_point1_value
pop_5[parent2][crossover_point2] = org_parent1_point2_value
if Constraints_more(pop_5[parent1]) and Constraints_s(pop_5[parent1]) and
Constraints_light(pop_5[parent1]) and Constraints_rebult_money(pop_5[parent1]) and
Constraints_more(pop_5[parent2]) and Constraints_s(pop_5[parent2]) and
Constraints_light(pop_5[parent2])and Constraints_rebult_money(pop_5[parent2]):
    return pop_5
else:
    return pop_4

# %%
def mutation(pop_7, parent1_num,mutation_rate):
    pop_8 = copy.deepcopy(pop_7) # 创建副本，避免直接修改原种群

    # 如果随机数小于变异率，则不进行变异，直接返回
    if random.random() > mutation_rate:
        return pop_7

    # 进行变异：选择一个随机的基因位置
    mutation_point = random.randint(0, len(pop_8[0]) - 1)

    # 变异为一个随机的地块编号
    # pop_8[parent1_num][mutation_point] = random.randint(0, 371)
    pop_8[parent1_num][mutation_point] =
random.sample(res_move_all_possibility[mutation_point],1)[0]

    # 如果不满足约束条件，重新变异
    if Constraints_more(pop_8[parent1_num]) and Constraints_s(pop_8[parent1_num]) and
Constraints_light(pop_8[parent1_num]) and Constraints_rebult_money(pop_8[parent1_num]):
        return pop_8
    else:
        return pop_7

# %%

```

```

# 在 crossover_and_mutation 函数中，确保为 select 传递 fitness 参数
def crossover_and_mutation(pop_3, crossover_rate, mutation_rate):
    # parent1,parent2 = random.sample(range(0,50),2)
    # parent2 = random.randint(0,49)
    parent1 = 48
    parent2 = 49
    #交叉操作
    pop_6= crossover(pop_3, parent1, parent2, crossover_rate)
    #变异操作
    pop_end = mutation(pop_6,parent1,mutation_rate)
    return pop_end

# %%
# #导入一个已经初始化的数据
df_org = pd.read_excel("pop_333.xlsx")
df_org_list = df_org.values.tolist()
pop_111 = df_org_list.copy()

# pop_111 = [[0]*113]*50

max_fit = []
for generation in range(N_GENERATIONS): # 迭代 N 代
    pop_111=multi_sort(pop_111)
    # 交叉和变异操作
    pop_111 = crossover_and_mutation(pop_111, CROSSOVER_RATE, MUTATION_RATE) # 交叉
    和变异生成新种群
    print(f'迭代到第 {generation+1} 次')
# %%
#输出数据
#1.输出 3 个函数的数据，也就是 50*3 的数据
function_output = [['满意度函数结果','搬迁数量函数结果','完整院落函数结果']]
for e in pop_111:
    tl_nnn_data = translateDNA(e)
    temp_o = []
    temp_o.append(F_res_satisfie(tl_nnn_data))
    temp_o.append(F_movenum(tl_nnn_data))
    temp_o.append(F_fullYard(tl_nnn_data))
    function_output.append(temp_o)
function_output_df = pd.DataFrame(function_output)
function_output_df.to_excel("多目标种群结果（函数）-40000.xlsx")
# %%
#2.输出 50 个个体的数据
pop_111_df = pd.DataFrame(pop_111)
pop_111_df.to_excel("多目标 50 个体搬迁数据-40000.xlsx")

```


问题二：绘制帕累托前沿曲面

```
clc; clear;
% 加载数据
data = xlsread('多目标种群结果（函数）-50000.xlsx');

n = size(data, 1);

% 帕累托筛选（全部最大化）
is_dominated = false(n, 1);
for i = 1:n
    for j = 1:n
        if all(data(j,:) >= data(i,:)) && any(data(j,:) > data(i,:))
            is_dominated(i) = true;
            break;
        end
    end
end

pareto_points = data(~is_dominated, :);

% 提取三个目标
x = pareto_points(:,1);
y = pareto_points(:,2);
z = pareto_points(:,3);

% 清洗掉含有 NaN 或 Inf 的点
valid_idx = isfinite(x) & isfinite(y) & isfinite(z);
x = x(valid_idx);
y = y(valid_idx);
z = z(valid_idx);

% 构造插值函数（基于散点）
F = scatteredInterpolant(x, y, z, 'natural', 'none');

% 构造网格
[xq, yq] = meshgrid(linspace(min(x), max(x), 50), ...
                    linspace(min(y), max(y), 50));

% 计算插值
zq = F(xq, yq);

% 绘图
figure;
hold on;
```

```

grid on;

% 原始帕累托点
scatter3(x, y, z, 50, 'r', 'filled');

% 插值后绘制曲面
surf(xq, yq, zq, 'EdgeColor', 'none', 'FaceAlpha', 0.5, 'FaceColor', [0.5647, 0.6667, 0.8588]);

xlabel('目标 1');
ylabel('目标 2');
zlabel('目标 3');
title('帕累托前沿三维曲面拟合');
legend('帕累托点', '拟合曲面');
view(135,30);
问题二：绘制帕累托最优解集合
clc; clear;

% 加载数据
data = xlsread('多目标种群结果（函数）-50000.xlsx');
% 获取个体数量
n = size(data, 1);
% 初始化支配标记
is_dominated = false(n, 1);

% 帕累托筛选
for i = 1:n
    for j = 1:n
        if all(data(j,:) >= data(i,:)) && any(data(j,:) > data(i,:))
            is_dominated(i) = true;
            break;
        end
    end
end

% 提取帕累托前沿个体（非支配解）
pareto_points = data(~is_dominated, :);

% 绘图
figure;
scatter3(data(:,1), data(:,2), data(:,3), 30, 'k', 'filled'); hold on;
scatter3(pareto_points(:,1), pareto_points(:,2), pareto_points(:,3), 60, 'r', 'filled');

grid on;
xlabel('目标 1');

```

```
ylabel('目标 2');  
zlabel('目标 3');  
title('三维帕累托解集');  
legend('所有个体','帕累托前沿','Location','best');  
view(135,30); % 调整视角
```